

# NBAccess/ClaudeRuntime/ClaudeOrchestratorとSourceVaultに関する技術調査レポート

調査基準日: 2026年7月16日（改訂版: 2026年7月17日）

## エグゼクティブサマリ

本調査対象の四つの公開GitHubリポジトリ、NBAccess、ClaudeRuntime、ClaudeOrchestrator、SourceVaultは、Wolfram Language/Mathematicaを実行基盤とする階層化システムを構成している。NBAccessがNotebook・機密性・安全実行の最終境界、ClaudeRuntimeが単一LLMターンとExpression-Proposalループ、ClaudeOrchestratorが複数workerとworkflow・single committer、SourceVaultがsource lifecycle・evidence・検索・長期知識を担当する。

| 層                  | 中心的責務                                                  | 代表的な境界                                             |
|--------------------|--------------------------------------------------------|----------------------------------------------------|
| NBAccess           | Notebook semantic API、privacy、held expression検証、暗号鍵    | 実行と秘密の最終境界                                         |
| ClaudeRuntime      | 単一-agent turn、provider adapter、外部runner、task placement | proposalを命令として直接評価しない                              |
| ClaudeOrchestrator | planning、worker、Petri-net workflow、reduce、commit       | workerはartifact producer、Notebookはsingle committer |
| SourceVault        | source/snapshot/claim、URI、検索、MCP、Knowledge Home        | release gateとprovenanceを維持した知識共有                   |

ただし、実際の利用面ではこの四層だけを見れば十分ではない。前面に出にくいのが、transreal/claudecodeにはClaudeEval、複数provider統合、Claude Directives、共有polling tick、palette、MCP server登録など多数の基本関数が残っており、現在の統合環境を支える互換・接着層として重要である。ClaudeRuntime/ClaudeOrchestratorへの責務移行が進んでも、現状の運用を理解するにはclaudecodeを背景基盤として考慮する必要がある。

利用経路も「ClaudeEvalでプロンプトからWolfram式を提案し、NBAccessで検証して実行する」だけではない。SourceVault PromptRouterは、保存済みまたは決定論的なPromptRouteをLLM呼び出し前に解決し、ClaudeEvalへ未評価式を提案できる。複雑なプロンプトはClaudeOrchestrator PromptWorkflowがWorkflowNet候補として扱い、新規workflowは安全なパースと検証を経てもNeedsApprovalで停止する。さらにSourceVaultには、Notebook上の要件から設計仕様を合意形成するspec-reviewと、承認仕様からコード化workflowを生成・検証するspec-implが実装されており、「要件→仕様→実装→検証→catalog登録」という開発自体をworkflow化している。

SourceVaultは、取り込んだsourceを単なるファイルパスではなく、外部API・検索結果・関係記述ではsv://...形式の正準URIで参照する方向へ統一している。mail/web/pdfなどはURI namespaceではなくClass/MediaType/Kindのsidecar metadataとして表現され、object、snapshot、artifact、chunk、file、packageapiなどのidentity namespaceを使う。旧来のsourceid、snapshot:...、blob:...等の内部参照は互換のため残るが、MCPやOrchestratorを跨ぐ交換ではsv:// URIが安定したopaque referenceとなる。

このURI層を外部LLMへ安全に開くのがSourceVault MCPである。現行MCPは26個のtoolを持ち、catalog/search/get、access request、runtime capability、Web検索、メール/OOPS検索、GitHub commit log、Claude Directivesのrules/skills参照、append-only artifact depositなどを提供する。Claude CodeやCodexはローカルharnessに展開されたClaude Directivesを使える一方、同じrules/skillsをMCPのデータベースとして公開することで、MCP対応のLM Studio等のローカルモデルも同じ規則・技能集合を参照できる。さらにLLMはsourcevault\_depositを通じてsv://artifact/...を作成できるため、SourceVaultをLLM/worker間のartifact交換面として利用できる。これはworkerがlive Notebookへ直接書き込めないsingle-committer原則と両立する。

自律実行面では、SourceVault\_autotriggerがschedule、SourceVault event、conditionを監視してjobを構築し、workflowやCatalogWorkflowを非同期に起動できる。設計仕様では、自動実行の最初のworkflowをSourceVaultSystemDoctor/diagnosticsによる包括的health checkとし、一般workflowは必要componentのhealthがOKまたは許容済みDegradedであることを前提とする。現行実装ではlightweight doctorによるdispatch gate、machine heartbeat、specific-machine dispatch、remote result publicationが実装されている一方、comprehensive doctorの自動triggerとWorkerPoolの完全な自動負荷分散/failoverは未完成である。

複数PCが同一SourceVaultを共有する構成では、per-machine single-writerのjobs/runs log、machine heartbeat、aggregator rollup、SpecificMachine placement、atomic dispatch claim、headless dispatch、ライセンス枠ガードを組み合わせること

で、特定PCへのリモートworkflow実行と結果共有がすでに可能である。これらは冗長化・負荷分散の基礎を形成するが、health-awareなWorkerPool自動選択や完全failoverは設計段階を含むため、実装済み機能と将来像を区別して評価する必要がある。

以上を踏まえると、四つの主要リポジトリとclaudecodeの補助層は、単にLLMをNotebookから呼び出すツール群ではない。現在は、知識の取得、URIによる参照、構造化、検索、マイニング、LLM推論、workflow生成、自律起動、複数PC実行、監査を、privacyとcapabilityの境界を維持しながら連結するWolfram Notebook向けのlayered operating modelへ発展している。

```
主 な コ ー ド ・ 文 書 根 拠 : SourceVault_info/docs/user_manual.md / SourceVault_info/docs/api_promptrouter.md /  
SourceVault_info/docs/api_mcp.md / SourceVault_info/design/sourcevault_universal_mcp_access_spec_v2.md /  
SourceVault_info/design/sourcevault_auto_trigger_scheduler_spec_v0_1.md / claudecode_info/docs/api.md
```

# 本レポートの読み方 — 用語・成熟度・正本

本レポートは対象システム固有の用語を高密度に使うため、本文に入る前に、(1) 実装成熟度ラベル、(2) 記述の正本、(3) 最小限の用語辞書、(4) 層を跨いで登場するデータ型の対応を先にまとめる。

## 成熟度ラベルと記述の正本

実装状況の記述には、次の四段階のラベルを用いる。

| ラベル    | 意味                                                  |
|--------|-----------------------------------------------------|
| 【実装済】  | 現行ソースコードに実装が確認できる（静的照合であり、本調査でtestを再実行した動作試験を意味しない） |
| 【部分実装】 | 一部経路のみ実装済みで、全面的な結線・強制は未了                            |
| 【設計】   | design仕様には存在するが、現行コードでは未実装または未結線                    |
| 【将来提案】 | 仕様にも未確定で、本レポート筆者による整理・提案                            |

記述の正本は、現行ソースコード（各 \*.wl）を第一、\*\_info/docs/api\*.md のAPI文書を第二とし、\*\_info/design/ 配下の仕様は将来方針・設計意図の確認に用いた。ソースとAPI文書が矛盾する箇所は本文に「原本間差分」として明示し、付録Cに一覧を置く。tool数、entrypoint数などの数値は2026年7月16～17日時点にソースから機械集計した確認値である。一方、本文中のテストgreen数（test pass数。「262 green」等）はAPI文書の記載値の引用であり、本調査でtestを再実行して得た値ではない。

## 用語ミニ辞書

| 用語                                                         | 最小限の説明                                                                                             |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| artifact / live state / single committer                   | workerが生成する中間成果物と、Notebook等の正本状態の区別。live stateへの反映は唯一のcommitterに限定する原則                             |
| Notebook semantic API / Held Expression                    | Notebookを文字列でなくCell式・metadataとして扱うAPI。Held ExpressionはHoldComplete等で評価を抑止したWolfram式                |
| turn / proposal / adapter                                  | 一回のLLM対話単位、LLMが返す未評価の提案、providerや実行方式をRuntime契約へ接続する関数集合（Association）                              |
| Petri net（place / token / transition）                      | workflowを状態（place）・状態を流れる値（token）・遷移で表す形式モデル。approval待ちやjoinを表現しやすい                                |
| source / snapshot / claim / chunk / span / Evidence Bundle | 原資料、その時点の不変版、抽出した主張、検索単位、本文内の位置参照、成果物が依存する根拠集合                                                     |
| provenance / staleness                                     | 生成物がどのsource・span・処理に由来するかの追跡。上流版の更新により根拠が古くなった状態がstaleness                                        |
| release gate / ReleaseContext                              | 検索・取得結果を返してよいかのpolicy判定と、その判定に使う登録済みprofile                                                        |
| sv:// URI / opaque reference                               | 本文を埋め込まない安定した参照identity。URI自体は権限を持たない。内部実装用のlegacy refとは区別する                                       |
| MCP / tool / projection layer / service kernel             | Model Context Protocol。toolはMCPが公開する操作。SourceVault MCPは内部objectの許可された投影だけを返す層で、service kernelが実行する |
| claudecode / Claude Directives / 共有polling tick            | 四層の外にある互換・接着パッケージ。DirectivesはCLAUDE.md・rules・skillsの正準集合。polling tickはFEを固めない共有監視基盤                |
| PrivacyLevel / AccessLevel / PrivacySpec                   | データ側の機密度（0.0～1.0）、送信先が受け取れる上限、その設定Association。判定はPrivacyLevel <= AccessLevel                       |
| task placement（main kernel / subkernel / wolframscript）    | 実行場所の区分。Notebook UI依存・秘密・license seat・隔離性が異なる                                                      |

| 用語                                                                                | 最小限の説明                                                                                            |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| capability / CapabilityLease / PreparedInputToken / ExecutionTicket / AccessGrant | 副作用の種類ラベル、期限付きの実行許可、LLM送信内容全体の束縛token、lease消費と同時に発行される一回限りの実行証明、データaccessの承認                      |
| immutable / append-only / event / current projection                              | event-sourcedとして実装された領域では記録を上書きせず追記し、現在状態をevent列から投影する。registry・sidecar・cache等には別の永続化契約もある（6.4参照） |
| taint / quarantine / declassification / sanitized projection                      | 危険入力への派生物への伝播追跡、隔離、制約付きの解除、安全化された公開用投影                                                            |
| retrieval / grounding / primer / BM25 / dense / rerank                            | 候補取得、原文への根拠付け、候補絞り込み用の要約層、語彙ベース検索、ベクトル検索、再順位付け                                                    |
| KG / Neighbor Hopping / TopicPosition / UnknownMass                               | knowledge graph、資料から資料への近傍移動、資料のtopic空間内の位置、既存topicで説明できない残余                                      |
| Knowledge Home / OOPS                                                             | 個人知識の中核作業面と、その初期座標となる1992～2005年の私的メーリングリストコーパス（固有名）                                               |
| SystemDoctor (lightweight / comprehensive) / SIEM                                 | dispatch前の軽量health gateと包括診断。SIEMはセキュリティイベントの集約・分析層                                               |
| WorkerPool / SpecificMachine / EnvironmentIndependent                             | 実行PC選択のmode。SpecificMachineは実装済み、WorkerPoolの自動負荷分散は未実装                                            |
| Shadow / Warn / Enforce                                                           | 観測のみ・警告して続行・拒否の段階mode。modeとは別に観測トグルがあり、両者は独立に設定される                                                |
| bitemporal / crypto-shredding / Kleene 3値論理                                       | 有効時刻と記録時刻の二重管理、鍵破棄による実質消去、True/False/Unknownの三値論理                                                 |
| owner / authority / principal                                                     | システムの管理者本人、命令として扱ってよい権限、認証された呼出主体                                                                 |
| sidecar metadata                                                                  | 本体record（URIやsnapshot）とは別ファイル・別キーで保持する補助metadata。媒体種別や解決状態などをURI本体へ埋め込まずに持つ                       |
| harness (local/tool harness)                                                      | CLIやworkerがfilesystem・shell・Directives・MCP toolを使うためのローカル実行環境一式                                   |
| headless                                                                          | FrontEnd/Notebook UIを使わないkernel・CLI実行                                                             |
| CatalogWorkflow                                                                   | catalog登録済みworkflowをslug等で解決して起動するAutoTriggerの実行種別                                                |
| DAG                                                                               | 有向非巡回グラフ。本レポートでは主にTaskSpecの依存関係と並列実行可能性を表す                                                        |
| green                                                                             | test passの意味。「262 green」はAPI文書記載のtest pass数                                                       |
| schema-only                                                                       | 値本文を除外し、型・shape・key・要素数だけを渡すcontext projection                                                    |
| fail-closed                                                                       | context不足や判定失敗時に許可へ倒さず、空結果または拒否へ倒す方針                                                              |
| TOCTOU                                                                            | time-of-check/time-of-use。検査時と使用時の間で対象が差し替わる問題                                                    |
| idempotency key                                                                   | 同一要求の再送を重複実行せずreplayとして扱うための識別子                                                                   |

なお、本文に登場する `$ClaudeAdvisaryModel` は綴りどおりの実在シンボル名であり（Advisoryの誤記ではない）、仕様レビューの検証モデルを指す。

## 層を跨ぐデータ型の対応

| 系統                    | 型                                                                                     | 所属層                                                                 |
|-----------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| privacy/access policy | PrivacySpec (AccessLevel含む) / AccessSpec・AccessRequest / ReleaseContext / AccessGrant | NBAccess / NBAccess・MCP / SourceVault searchindex / SourceVault MCP |
| 実行権限token             | CapabilityLease / PreparedInputToken / ExecutionTicket                                | SourceVault capbroker                                               |
| タスクと成果物               | TaskSpec / Artifact / ReducedArtifact                                                 | ClaudeOrchestrator                                                  |
| 知識の単位                 | source / snapshot / object / chunk / claim / span / artifact                          | SourceVault                                                         |

これらは同じ「絞る・束ねる」発想を層ごとに独立実装したものであり、単一の共通policy型がすべてを統合しているわけではない（第6章参照）。

## 第1章 調査対象とシステム全体像

---

本章では、調査対象である四つのリポジトリの範囲を確定し、それらが構成する階層アーキテクチャの全体像を示す。NBAccess、ClaudeRuntime、ClaudeOrchestrator、SourceVaultは、それぞれ単独のツールとして理解するよりも、Notebookを起点にLLM推論、workflow実行、知識保存、監査を分担する一つの階層システムとして捉えた方が理解しやすい。この設計の狙いは、危険な副作用や機密情報に近い責務ほど下位層へ置き、上位層ほど計画・編成・知識利用へ集中させることで、機能追加が安全境界を侵食しない構造を作ることにある。四つのリポジトリは同じ機能を重複して実装しているのではなく、この責務勾配に沿って役割を分担している。

また、現行運用を理解するには四つのリポジトリだけでなく、claudecodeを背景の統合・互換レイヤとして見る必要がある。ClaudeEval、provider選択、Claude Directives、共有polling tick、palette、MCP server registryなど、歴史的にclaudecodeへ蓄積された基本機能が現在も四層を接着している。したがって本章では、「論理アーキテクチャとしての四層」と「移行期の実装基盤としてのclaudecode」を区別して読むことが重要である。

利用形態も一問一答のClaudeEvalだけではない。PromptRouterによる決定論的ルート、PromptWorkflowによる複雑タスクのworkflow化、spec-review/spec-implによる要件から仕様・実装への展開、MCPによる外部LLMとの共有、AutoTriggerによる自律起動まで含めると、システムはNotebook操作ツールから知識・実行基盤へ広がっている。以下、1.1～1.3で調査対象と論理アーキテクチャを、1.4～1.5で移行期の実装基盤と利用経路の広がり述べ、1.6で今後の展望をまとめる。

### 1.1 調査対象

対象となるリポジトリは次の四つである。

- transreal/NBAccess
- transreal/ClaudeRuntime
- transreal/ClaudeOrchestrator
- transreal/SourceVault

当初調査時点では、各READMEにMIT Licenseが記載され、デフォルトブランチはmainであった。

規模と性格は大きく異なる。NBAccessは比較的小さな基盤パッケージである。ClaudeRuntimeは単一エージェントの実行核に、外部実行、process supervision、seat管理、task placementなどを付加した構成を持つ。ClaudeOrchestratorはworkflow、observability、PromptWorkflow、sessionを含む上位制御層である。SourceVaultは多数のサブパッケージを持つ最大のリポジトリであり、source管理、検索、Web、メール、identity、マイニング、サービス管理などを包含する。

### 1.2 四層アーキテクチャ

基本的な依存関係は図1のように表せる。

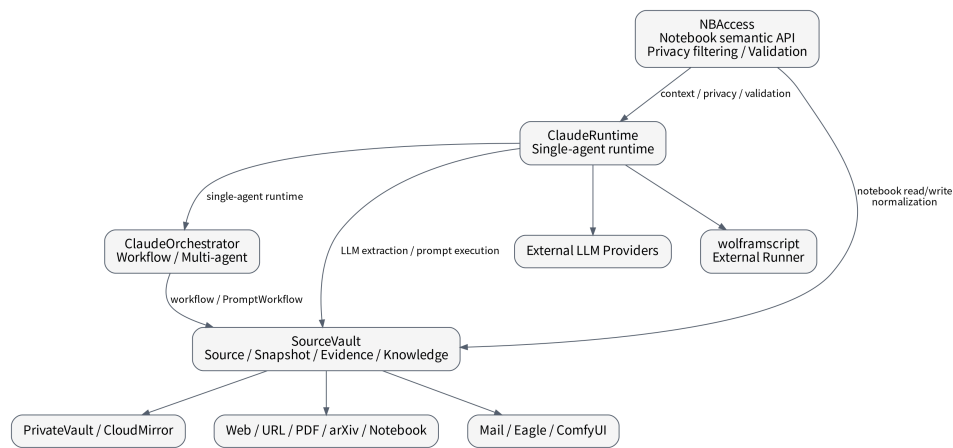


図1 四層アーキテクチャと外部システムの関係

この図で重要なのは、ClaudeRuntimeがNotebookやsecretの意味論を持たず、adapterを介してNBAccessを呼び出す点である。また、SourceVaultはsource lifecycleを管理するが、式の安全な実行はNBAccess、複数workerの統制はClaudeOrchestratorへ委譲する。

実際のシステムは単純な一方向の積層ではなく、図2のような循環構造を持つ。

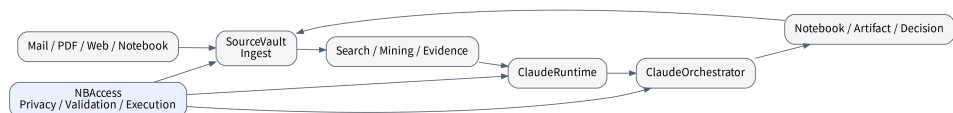


図2 SourceVaultを中心とした知識・実行の循環

SourceVaultから取得されたevidenceがRuntime／Orchestratorへ渡り、生成されたNotebook、コード、レポート、decision、workflow logがSourceVaultへ戻る。これにより、作業結果は次の探索や推論の入力となる。

### 1.3 想定される主要ユースケース

四リポジトリが想定する主要用途は、次の四種類に大別できる。

第一は、機密NotebookをローカルLLMとクラウドLLMへ安全に振り分けながら操作することである。

第二は、複雑なタスクを複数のsubtaskへ分解し、workerを実行し（既定は直列、設定により依存関係を保ったDAG並列）、成果物を検証して一つのNotebookへcommitすることである。

第三は、URL、PDF、arXiv、Notebook、メールなどをimmutable snapshotとして保存し、claim、evidence、成果物との関係を管理することである。

第四は、頻出する自然言語指示を、決定的な関数呼び出しや未評価式へ変換し、同じ意味の処理を再現可能にすることである。

この時点でも一般的なRAGやagent frameworkよりNotebook中心に最適化された設計であるが、後述するマイニングと安全レイヤーにより、さらに長期知識基盤としての性格が強くなっている。

### 1.4 claudecode — 前面に出ない統合・互換レイヤ

本レポートの主対象は四つのリポジトリであるが、現行コードを正確に理解するにはtransreal/claudecodeの存在を無視できない。claudecodeはWolfram Language／MathematicaからClaude Code CLI、Codex CLI、Anthropic/OpenAI API、LM Studioなどへ接続する統合インターフェースを提供し、ClaudeEval、provider/model選択、Notebook側UI、共有polling tick、外部package API文書の条件付き注入、CLI向けMCP server registryなど、多数の基本機能を保持している。



| claudecode側の代表機能                                                                                    | 四層との関係                                               |
|-----------------------------------------------------------------------------------------------------|------------------------------------------------------|
| ClaudeEval / ContinueEval                                                                           | Runtimeのproposal loopやPromptRouterへ入るNotebook側入口     |
| <code>\$ClaudeModel</code> / <code>\$ClaudeAdvisaryModel</code> / <code>\$ClaudePrivateModel</code> | 通常・検証・privateモデルの選択                                  |
| Claude Directives                                                                                   | Claude Code/Codex/workerへrules・skillsを投影             |
| ClaudeRegisterPollingTick系                                                                          | diagnostics、auto-trigger、background job監視の共有tick基盤   |
| <code>\$ClaudeCLIMCPServers</code>                                                                  | SourceVault MCP等をheadless CLI harnessへ登録する中立registry |
| palette / FE連携                                                                                      | 仕様生成・実装、job進捗、結果書き戻しの既存UI接着層                         |

したがって、論理アーキテクチャ上はNBAccess/Runtime/Orchestrator/SourceVaultの責務分離を正準としつつ、実装移行期の実運用ではclaudecodeがbootstrap（package初期化・依存読み込み・registry構築）・UI・polling・provider互換を支える「背景の接着層」として機能している、と整理するのが適切である。

【将来提案】将来整理の想定としては、肥大化したclaudecode.wlをコア機能と互換性維持のlegacy層へ分離し、ClaudeOrchestrator / ClaudeRuntime / claudecode\_core / NBAccessを中心とする運用へ移行する構成が自然である。ただし、2026年7月16日時点の公開リポジトリでclaudecode\_core分離が実装済みであることは確認できず、これは確定ロードマップではなく本レポート筆者による整理案である。

## 1.5 利用形態は「一問一答」から「知識・workflow・service」へ拡張

四層を利用者の視点から見ると、入口は少なくとも五種類ある。第一はClaudeEvalによる対話的な式提案である。第二はPromptRouterによる決定論的な高レベルAPI呼び出しである。第三はClaudeOrchestrator/SourceVault workflowによる長時間・複数段階の処理である。第四はMCPを介した外部LLMクライアントからの利用であり、第五はAutoTriggerによる、Notebookを開いていない時間も含むservice的な自律処理である。

| 入口           | 典型例                                         | 実行主体                                |
|--------------|---------------------------------------------|-------------------------------------|
| ClaudeEval   | 「このNotebookを要約して」「予定を表示して」                  | claudecode → Runtime → NBAccess     |
| PromptRouter | 保存済みRoute、UpcomingSchedule、FindNotebooks    | SourceVault route → held expression |
| Workflow     | 要件→仕様、仕様→実装、調査→検証→commit                    | Orchestrator / SourceVault workflow |
| MCP          | LM Studioから検索、Directives参照、artifact deposit | MCP service kernel + adapters       |
| AutoTrigger  | 時刻/eventでworkflow起動、特定PCへdispatch           | SourceVault scheduler + executor    |

主 な コ ー ド ・ 文 書 根 拠 : [claudecode\\_info/docs/api.md](#) / [SourceVault\\_info/docs/user\\_manual.md](#) / [SourceVault\\_info/docs/api\\_workflowregistry.md](#)

## 1.6 今後の展望

今後のアーキテクチャ上の焦点は、責務分離をさらに明示し、移行期の接着コードを整理することである。特にclaudecode.wlに残る機能を、恒久的なcoreと互換性維持のlegacyへ分け、ClaudeRuntime、ClaudeOrchestrator、NBAccess、SourceVaultとの境界をAPI契約として固定できれば、システム全体の理解と保守が容易になる。

もう一つの焦点は、四層を「ライブラリ群」ではなく「一つの運用モデル」として観測できるようにすることである。どの入口から要求が入り、どのprivacy decisionを通り、どのLLMとworkflowを経て、どのsv:// objectやNotebook cellへ結果が戻ったのかを一つのrun traceとして追跡できれば、設計上の責務分離が運用上の可観測性にも反映される。

長期的には、Notebook中心の対話環境、headless service、MCP client、複数PC workerが同一のpolicy・identity・provenanceモデルを共有する方向が自然である。ただしこれは【統合目標】であり、現行は層ごとに異なるpolicy語彙とregistryが併存する

(第6章参照)。その際も、上位層が下位層の権限を拡張できないという原則を維持することが、システムの独自性を保つ鍵になる。

## 第2章 NBAccess — Notebookと機密の最終境界

NBAccessは、四層の最下位でNotebook操作、機密性、アクセス制御、安全な式実行を担う最終境界である。RuntimeやOrchestratorがどれだけ複雑な計画を立てても、Notebookの機密情報をどこまで見せるか、提案されたWolfram式を本当に実行してよいか、秘密鍵をどのように扱うかはNBAccess側で決める。この設計の狙いは、安全判断の正本を一箇所に集約し、上位層の設定や便利機能によって権限が拡張される経路を構造的に断つことにある。したがってNBAccessを理解すると、上位層がなぜ自由にNotebookを操作できない設計になっているかが分かる。

本章で特に重要な機構は二つある。第一は、privacyが「秘密セルそのもの」だけに付くのではなく、そのセルから導出された変数や出力へ伝播する点である。値を直接公開していなくても、秘密値から計算された結果が情報を漏らす可能性があるため、Notebook contextをLLMへ渡す前に依存関係を解析し、本文を渡すかschema-onlyにするかを決める。この考え方はSourceVaultのprivacy propagationやMCP release gateにもつながっている。第二は、LLM出力を命令ではなくHeld Expressionとして扱うことである。提案式はNBValidateHeldExpr等で評価前に検査され、Permit、NeedsApproval、Deny、RepairNeededといったdecisionへ分類される。これはPrompt Injectionや誤生成への対策でもあり、「LLMがそう言ったから実行する」という経路を構造的に排除する。

以下、2.1～2.2で役割とAPIの全体像を、2.3～2.6でprivacy伝播・ファイル境界・安全実行・暗号という各機構の詳細を述べ、2.7で評価、2.8で今後の展望をまとめる。

### 2.1 役割と主な責務

NBAccessは、Mathematica Notebookをセルの配列として扱い、セル単位の読み書き、privacy評価、依存解析、履歴管理、安全な式実行を提供する基盤層である。Notebook内の情報をLLMへ渡す際に、単にNotebook全体を文字列化するのではなく、各セルのprivacy levelと依存関係を評価する点が中心的特徴である。

privacyモデルでは、二つの値を区別する。データ側の機密度を表すPrivacyLevel（0.0～1.0。既定では0.0=公開、0.75=秘匿依存、1.0=秘匿の三段階が中心）と、送信先が受け取ってよい上限を表すAccessLevelである。アクセス可否の実装は `NBIsAccessible` の `PrivacyLevel <= AccessLevel` 判定であり、既定のプライバシースペックおよびクラウドprovider（例：Anthropic）の上限はAccessLevel 0.5であるため、PrivacyLevel 0.5ちょうどのセルまでが許可され、0.5を超えるセルが除外される。高いprivacyを持つセルや、それに依存して生成された出力は、クラウド向けcontextから除外される。

| 責務                       | 内容                                 |
|--------------------------|------------------------------------|
| Notebook semantic access | セルの取得、位置、型、スタイル、内容の統一的操作           |
| Privacy filtering        | セルごとのPrivacyLevelとAccessLevelによる選別 |
| Dependency propagation   | 秘密変数へ依存する出力への機密性伝播                 |
| Context projection       | 本文、要約、schema-onlyなど用途別contextの生成   |
| Path/value normalization | SourceVault向けのobject spec生成        |
| History                  | Notebook変更履歴と差分                    |
| Safe execution           | 保持された式の検証、承認、実行                    |
| Crypto                   | KeyRefによる暗号化、MAC、鍵隔離               |

主要ファイルは次のとおりである。

| ファイル               | 主な役割                         |
|--------------------|------------------------------|
| NBAccess.wl        | Notebook操作、privacy、依存解析、安全実行 |
| NBAccess_crypto.wl | KeyRefベース暗号化、MAC、鍵管理         |
| NBAccess_info/     | 設計・APIなどの補足資料                |

## 2.2 主要API

| API                          | 役割                                    |
|------------------------------|---------------------------------------|
| NBGetContext                 | privacy filterを適用したNotebook context取得 |
| NBBuildGlobalVarDependencies | 変数依存関係の構築                             |
| NBExtractAssignments         | 代入式の抽出                                |
| NBFileSpec                   | ファイルをSourceVault向けobject specへ変換      |
| NBNormalizePath              | パスの正規化                                |
| NBValueSpec                  | 値の構造情報をobject spec化                   |
| NBValidateHeldExpr           | 保持式の安全性判定                             |
| NBExecuteHeldExpr            | 許可された保持式の実行                           |
| NBAuthorize                  | policyに基づく実行許可判定                      |
| NBGenerateSymmetricKeyRef    | 対称鍵を生成しKeyRefを返す                      |
| NBEncryptWithKeyRef          | KeyRefを使った暗号化                         |
| NBVerifyMacWithKeyRef        | MAC検証                                 |

## 2.3 Privacyと依存性伝播

秘密情報の保護では、秘密セルそのものを除外しても十分ではない。

```
Secret Cell
  ↓
secretVariable
  ↓
derivedOutput
```

derivedOutputに秘密値が直接書かれていなくても、それがsecretVariableから計算された結果なら、秘密情報を推測できる可能性がある。NBAccessはNotebook中の代入、変数参照、出力依存関係を解析し、機密性を伝播する。そのうえで、送信先や用途に応じたcontextを構築する。

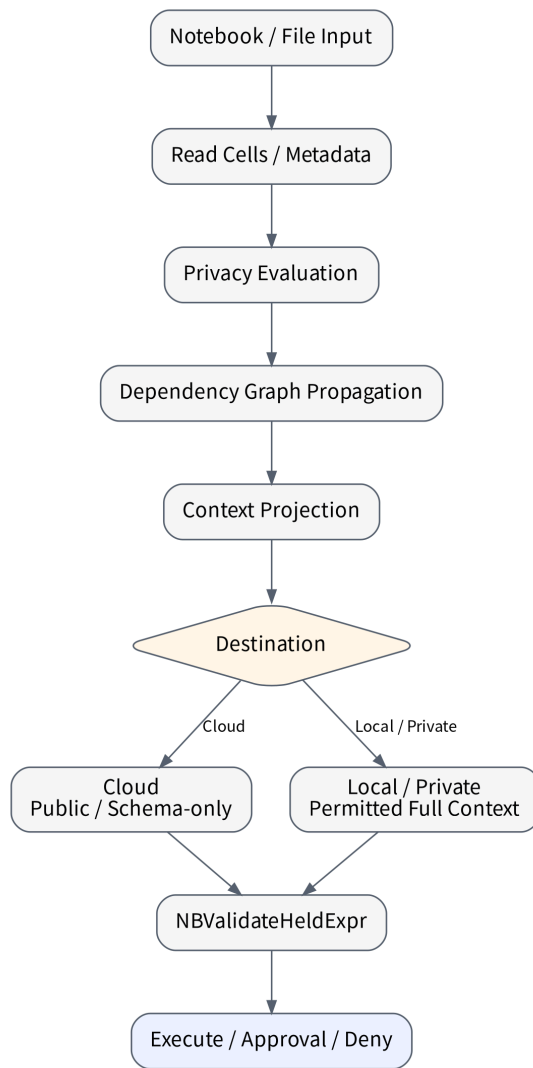


図3 NBAccessにおけるprivacy評価・context projection・安全実行

秘密依存outputについては、本文を送らず、型、サイズ、key、要素数などのschemaだけを送る選択肢がある。これにより、LLMへ計算構造を知らせつつ、値そのものを開示しないことができる。

## 2.4 ファイルprivacyとCloudPublishable境界

NBFileSpec周辺の動作は、概念的には次のように表現できる。

```

function NBFileSpec(path):
  if file does not exist:
    return Exists=False, PrivacyLevel=1.0

  ext = file extension

  if ext == ".nb":
    open notebook safely
    evaluate cell privacy
    count public/confidential cells
    propagate confidentiality through dependencies

    if notebook declares CloudPublishable=True:
      apply controlled public boundary
    if notebook declares CloudPublishable=False:
      PrivacyLevel = 1.0
  else:
    PrivacyLevel = file privacy policy result

  calculate content hash
  construct normalized path/value references
  cache base result
  reapply declared boundary after cache lookup
  return FileSpec

```

Notebook 側の宣言は TaggingRules の `{TaggingRules, "SourceVault", "CloudPublishable"}` に格納され、NBGetCloudPublishable / NBSetCloudPublishableで読み書きできる。重要なのは、キャッシュされた古いprivacy判定によって Notebook側の最新宣言が隠れないよう、base specのキャッシュ境界の後で宣言を意図的に再適用する点である。実装コメントにも「stale base specが明示的なCloudPublishable宣言を隠してはならない」と明記されている。

## 2.5 安全な式実行

LLMから返されたWolfram Language式を、そのままToExpressionなどで評価する設計ではない。

```

LLM Output
↓
HoldComplete[expression]
↓
NBValidateHeldExpr
↓
Permit / NeedsApproval / Deny / RepairNeeded
↓
NBExecuteHeldExpr

```

危険なhead、Notebookへの書き込み、外部プロセス、credential、ネットワークアクセスなどは、policyに基づいて分類される。なお、Permit / NeedsApproval / Deny / RepairNeededはNBValidateHeldExpr系の分類語彙であり、より上位のNBAuthorizeはPermit / Deny / Screen / RequireApprovalという別の語彙を持つ。両者は役割が異なるため混同しないよう注意が必要である。

NBAccessはこの最終判断を持つため、RuntimeやOrchestrator側の設定だけで権限を拡張できない。

## 2.6 暗号レイヤー

NBAccess\_crypto.wlは、鍵そのものではなくKeyRefを外部APIで扱う。概念的には次のように動作する。

```
function NBEncryptWithKeyRef(keyRef, plaintext):
    key = resolve internally
    if key type is not symmetric:
        fail

    encrypted = Encrypt(key, plaintext)

    if encryption failed:
        fail

    return serialized encrypted object
```

MAC検証では、通常の文字列比較ではなくconstant-time比較を使用する。実装は長さ確認後に全バイトをXORで畳み込む方式で、早期returnを持たない。

```
function NBVerifyMacWithKeyRef(keyRef, bytes, expectedMac):
    actualMac = calculate MAC using internal key
    return constant_time_equal(actualMac, expectedMac)
```

鍵材料は戻り値や通常ログへ出さず、KeyRef indexにはmetadataのみを保持する。backendは二種類が定義されており、既定のmemory backendは主として開発・テスト向け（kernel内のみ、同期されない）、SystemCredential backendが本番想定である。SystemCredential backendは、OS credential storeへの鍵材料の書き込み・読み出し・削除と、metadata indexの永続化・再読み込みまで【実装済】である。残る課題は「APIの実装」ではなく運用品質の成熟であり、具体的にはbackend書き込み失敗の伝播（現行NBStoreCredentialKeyはput失敗値を検査せず後続へ進む）、複数kernel間の競合、cross-kernel回帰試験である。なお原本間差分として、api\_crypto.mdとソース先頭コメントの一部にはなお「書き込みAPIはTODO」という古い記述が残っており、NBStoreCredentialKey／NBDeleteCredentialKeyの戻り値もAPI文書（keyRef／Boolean）と現行コード（Association）で食い違っている（付録C参照）。

## 2.7 評価

NBAccessは四リポジトリの中で最も明確なsecurity boundaryである。

長所は、Notebookセル、変数依存性、保持式、安全実行、暗号鍵が一つの境界として設計されていることである。

一方、この層に不具合があると上位全体へ影響するため、最も厳密な回帰試験、property test、境界値試験、cross-kernel testが必要なパッケージでもある。

## 2.8 今後の展望

今後のNBAccessでは、policy boundaryの形式化と回帰検証が最重要である。特にconfidential dependency propagation、CloudPublishable境界、held expressionのbypass耐性、Notebook semantic writeのatomicityは、上位全体の安全性を支えるため、property testやcross-kernel testを増やす価値が高い。暗号レイヤーでは、実装済みのSystemCredential backendについて、書き込み失敗の伝播、credential storeのOS差、複数kernel競合の検証と、API文書・戻り値契約の同期を完了させることが直近の課題である。

また、現在はNotebook中心のprivacyモデルを、sv:// object、MCP projection、remote workflow resultにも一貫して適用する必要がある。NBAccessがすべての媒体を直接理解する必要はないが、「最終的なNotebook書き込み」「secretを含むcontext

release」「外部副作用」という境界で同じpolicy snapshotを参照できると、SourceVault側のrelease decisionとより強く整合する。

長期的には、NBAccessのpolicyを宣言的なAccessSpecとして固定し、Runtime／Orchestrator／MCPが同じdigestを参照する形が望ましい。これにより、実行途中でpolicyが差し替わるTOCTOU問題や、processを跨いだpolicy driftをさらに減らせる。



## 第3章 ClaudeRuntime — Expression-Proposalループ

ClaudeRuntimeは、「プロンプトから式を提案し、検証を経て実行する」という一回の実行サイクルを管理する層である。NBAccessが最終的な安全判断を持つのに対し、RuntimeはLLMとの会話状態、provider呼び出し、proposal parsing、repair、継続判定を担当する。つまりRuntimeは安全性の正本ではなく、提案を安全境界へ運ぶ進行管理エンジンである。この分離の狙いは、provider・モデル・実行方式が変わってもturn進行の骨格を共通に保ち、どの入口から来た要求も最終的に同じheld-expression検証境界へ収束させることにある。

ここで理解しておきたいのは、ClaudeEvalの見かけ上の一回の呼び出しの背後に、BuildContext、QueryProvider、ParseProposal、ValidateProposal、DispatchDecisionという段階があることである。PromptRouterが決定論的な未評価式を返す場合も、通常のLLMが式を提案する場合も、最終的にはこのproposal contractへ収束させることで、入口ごとの差異を小さくしている。

Runtimeはさらに外部runner、process supervisor、seat broker、task placementを持ち、重い処理をmain kernelから分離する。これによりNotebookのFrontEndを止めずに長時間処理を行える一方、PID再利用、license seat、cleanup failureなど「LLMとは別種の運用上の失敗」も扱う必要がある。

以下、3.1～3.4でturn実行の骨格を、3.5で入口の多様性とPromptRouterとの境界を、3.6～3.7で実行場所の選択と外部プロセス管理を述べ、3.8で評価、3.9で今後の展望をまとめる。

### 3.1 役割

ClaudeRuntimeは、単一エージェントの一回または継続的なターンを管理するExpression-Proposalループ状態機械である。

基本フローは次の五段階である。

```
BuildContext
↓
QueryProvider
↓
ParseProposal
↓
ValidateProposal
↓
DispatchDecision
```

Runtime自身はNotebook、秘密情報、access policyの意味論を所有しない。それらはadapterとNBAccessへ委譲される。この責務分離により、Runtimeは「何を実行してよいか」ではなく、「提案をどの順序で処理し、どのdecisionへ進めるか」を担当する。

### 3.2 主要ファイル

| ファイル                               | 主な役割                                |
|------------------------------------|-------------------------------------|
| ClaudeRuntime.wl                   | RuntimeState、turn DAG、proposal loop |
| ClaudeRuntime_externalrunner.wl    | wolframscript外部実行                   |
| ClaudeRuntime_processsupervisor.wl | 外部プロセス監督                            |
| ClaudeRuntime_seatbroker.wl        | 実行seat管理                            |
| ClaudeRuntime_session.wl           | runtime session                     |
| ClaudeRuntime_sessionrunner.wl     | session実行補助                         |
| ClaudeRuntime_taskplacement.wl     | task分類とbackend推奨                    |

| ファイル                | 主な役割     |
|---------------------|----------|
| ClaudeRuntime_info/ | 設計・API資料 |

### 3.3 Adapterモデル

Runtime生成時には、adapterが必要な機能を提供する。multi-turn adapter（CreateClaudeRuntime）の契約は次の七つである。

```
BuildContext
QueryProvider
ParseProposal
ValidateProposal
ExecuteProposal
RedactResult
ShouldContinue
```

このadapter構造により、LLM provider、CLI、ローカルモデル、Notebook context、実行方式をRuntime本体から分離できる。なお、Orchestrator workflow から呼ばれる単一turnのTransition実行API `ClaudeRuntimeExecuteTransition[adapter, contextPacket]` は、このうちBuildContext、QueryProvider、ValidateProposal、ExecuteProposal、RedactResultの五つのみを要求する簡略契約であり、multi-turn adapterの公開契約とは適用範囲が異なる。

### 3.4 ターン実行とDecisionモデル

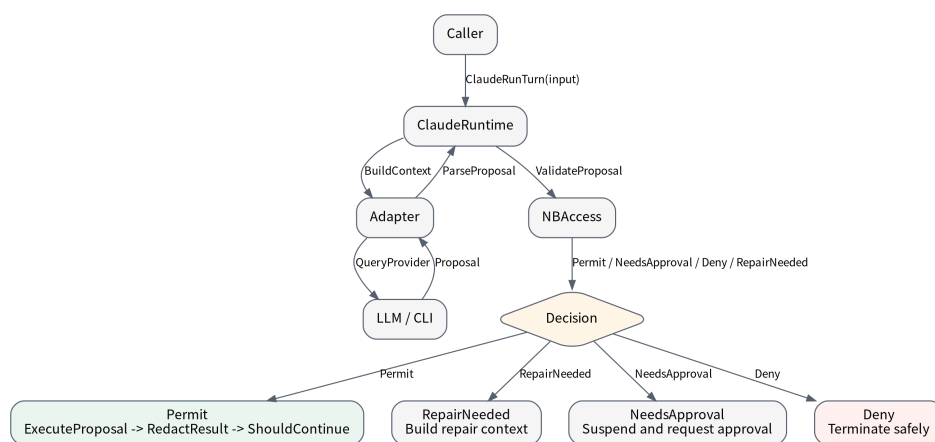


図4 ClaudeRuntimeのExpression-Proposalループ

ClaudeRunTurnは、buildContext、queryProvider、parseProposal、validateProposal、dispatchDecisionなどのノードをDAGとして構築し、jobを起動する。LLM応答を直接評価せず、proposalとして解析し、NBAccessによるvalidationを必須にする点が中核である。

decisionは二段階に分けて理解する必要がある。第一に、adapterのValidateProposal契約（NBAccess validation）が返すvalidation decisionは次の四つである。

| Validation decision | 意味            |
|---------------------|---------------|
| Permit              | 検証済みで実行可能     |
| NeedsApproval       | ユーザー承認が必要     |
| Deny                | 実行不可          |
| RepairNeeded        | proposal修正が必要 |

第二に、Runtimeはproposalの形そのものを見て、validationへ進む前にdispatch outcomeを生成する場合がある。

| Runtime dispatch outcome | 意味               |
|--------------------------|------------------|
| TextOnly                 | 式実行ではなく文章回答として処理 |
| ToolUse                  | 明示的tool経路へ進む     |

TextOnlyとToolUseはNBAccess validationの分類ではなく、Runtimeが生成するdispatch decisionである点に注意が必要である。この二段の分類により、「実行できない応答」をすべてエラーとせず、文章回答、修正、承認待ちなどへ分岐できる。契約違反時のRepairNeeded昇格や、Permit→NeedsApprovalへの昇格ロジックも実装されている。

### 3.5 入口の多様性 — ClaudeEvalとPromptRouterの境界

ClaudeRuntimeを理解する際に注意すべき点は、ClaudeEvalが「必ずLLMに自由生成させる関数」ではないことである。現行のNotebook側入口では、LLMを呼ぶ前に保存済みPromptRouteやSourceVaultの決定論的routeを提案できる。

SourceVaultProposePromptRouteは評価済み結果ではなくHoldComplete[...]に入った未評価式を返し、その式だけがRuntimeへ渡され、head allowlistとNBAccessの検証を経て実行される。たとえば「今日から3日間のスケジュールを」という定型要求は、毎回重量級LLMに解釈させる代わりに、SourceVaultUpcomingScheduleへの凍結された関数呼び出しへ変換できる。保存済みpromptが完全一致する場合も、SourceVaultProposeSavedPromptRouteがLLM呼び出し前に既存routeを提示できる。これにより、同じ要求を低コスト・高再現性で再実行しつつ、評価前検証というClaudeEvalの安全契約を維持する。

```
ClaudeEval[prompt]
↓
保存済みroute / FunctionRoute / lexical route
↓
未評価 HoldComplete[SourceVaultFunction[...]]
↓
Runtime head検査 + NBAccess validation
↓
実行

未解決の場合
↓
single-step Heavy LLM または PromptWorkflow候補
```

責務境界を整理すると、PromptRouterは「どの高レベル関数にルーティングするか」を解決するが、式の実行権限を持つauthorityではない。一方Runtimeは、providerとの対話、proposal解析、repair/approvalを含むturn進行を管理するが、Notebook privacyや最終実行権限を所有しない。この分離により、PromptRouterが決定論的に式を作った場合でも、LLMが自由生成した式の場合でも、最終的に同じheld-expression validation境界へ収束できる。

主 な コ ー ド ・ 文 書 根 拠 : [SourceVault\\_info/docs/api\\_promptrouter.md](#) / [SourceVault\\_info/docs/user\\_manual.md](#) / [claudecode\\_info/docs/api.md](#)

### 3.6 Task placement

ClaudeRuntime\_taskplacement.wlは、taskをどこで実行するかを補助的に判定する。

```

Raw Task
  ↓
ClaudeNormalizeTaskSpec
  ↓
ClaudeClassifyTask
  ↓
ClaudeSelectExecutionBackend
  ↓
ClaudeBuildTaskAction
  ↓
NBAccess / Orchestrator final decision

```

代表的な実行先は次のように整理できる。

| 実行先           | 適する処理                               |
|---------------|-------------------------------------|
| Main Kernel   | Notebook UI、FrontEnd依存、live state操作 |
| Subkernel     | 副作用のない数値計算、並列純粋計算                   |
| WolframScript | 長時間batch、外部I/O、大量ingest、独立process   |

最終的な許可はNBAccess policyとOrchestratorのworkflow policyで決まる。Runtimeの推薦だけで危険なbackendへ移行することはない。

### 3.7 外部プロセス管理

外部ランナーは、長時間処理をmain kernelから分離する。Process Supervisorは二段階manifestを用いる。

```

PendingSpawn （spawn前に永続化）
  ↓
OS process spawn
  ↓
PID + process start time 取得
  ↓
Running
  ↓
Completed / Reaped / Vanished （終了時。cleanup失敗はCleanupFailed）

```

PIDだけでなくprocess start timeを記録することで、OSによるPID再利用時に別プロセスを誤停止する危険を減らす。

終了時のcleanupはseat解放→archiveの順序が固定されており、seat解放に失敗した場合はarchiveせずCleanupFailedとして残置し、診断対象とする。archiveはcleanup手順の一段階（および格納ディレクトリ）であり、独立した状態名ではない。

### 3.8 評価

ClaudeRuntimeの最大の長所は、単一責任が明確であることだ。安全判断をNBAccess、複数agentの編成をOrchestrator、長期知識をSourceVaultへ委譲するため、Runtime本体はproposal loopに集中できる。

一方、adapter、legacy claudecode、外部runner、session、task placementの関係は複雑であり、ロード順、互換モード、fallback経路を明示的に管理しなければならない。

### 3.9 今後の展望

今後のRuntimeでは、単発turnだけでなくRuntimeSessionを公開ライフサイクルとして整理することが重要になる。現行にはConversationState、TurnCount、tool loop、checkpoint素材があるが、process再起動を跨ぐdurable checkpoint、budget suspend/resume、tool-call単位approvalなどはまだ完全なsession protocolとして統合されていない。

また、Prompt Injection対策の観点では、LLM送信直前のPreparedInputToken検証をRuntime/adapter境界で完全Enforceできるようにすることが重要である。どのproviderへ、どのmessages/tool schema/retrieval refsを送ったかをrequest envelope全体で固定し、送信直前に照合することで、context構築後の差し替えを防げる。

task placementについては、性能最適化だけでなくprivacy、license seat、GPU/CPU資源、SpecificMachine条件を含む実行契約へ拡張する余地がある。Orchestratorのresource placeと接続すれば、単一turnの実行場所選択から複数PCのresource schedulingへ自然に発展できる。

## 第4章 ClaudeOrchestrator — multi-agent編成とsingle committer

ClaudeOrchestratorは、複雑な要求を複数のworkerへ分解し、成果物を統合して最終commitまで進める層である。Planning、Spawn、Reduce、Commitという流れで見ると分かりやすい。Plannerが仕事をTaskSpecへ分解し、workerが個別のartifactを作り、Reducerが統合し、最後にsingle committerだけがNotebookなどのlive stateへ反映する。

この層の核心はsingle-committer原則であり、それは単なる実装上の都合ではなく安全性と再現性の中心である。workerが同じNotebookへ直接書き込むと、競合、順序依存、部分失敗、rollback困難が生じる。そこでworkerをartifact producerへ限定し、共有物はSourceVaultや明示的artifact refを介して渡す。MCP depositやsv://artifact/...は、この原則を維持したままagent間で成果物を共有するための自然な媒体になる。

さらにOrchestratorはPetri net workflowを用いるため、単純なDAGだけでは表しにくいapproval待ち、retry、複数branchのjoin、外部job完了待ち、cancel/recoveryを扱える。PromptWorkflowでは複雑な自然言語要求をworkflow候補へ変換でき、spec-review/spec-implでは「要件から仕様を作り、その仕様を実装・検証する」開発工程自体がworkflowになっている。

以下、4.1～4.4で役割とworker統制の基本構造を、4.5で実行パイプラインの各段階を、4.6～4.8でPetri net・PromptWorkflow・開発workflow化を述べ、4.9で評価、4.10で今後の展望をまとめる。

### 4.1 役割

ClaudeOrchestratorは、複数のRuntimeまたはworkerを上位から編成するmulti-agent orchestration層である。

基本フローは次の四段階である。

```
Planning
  ↓
Spawn
  ↓
Reduce
  ↓
Commit
```

ClaudeRuntimeが一つのproposal loopを扱うのに対し、ClaudeOrchestratorはタスク分解、依存関係、worker、成果物、commit orderingを管理する。

### 4.2 設計転換の核心 — workerとcommitterの分離

過去設計でworkerがlive Notebookへ直接書き込む構成には、複数workerによる共有状態の不整合、EvaluationNotebook[]の不安定な解決、CreateNotebookの意図しない実行、commit順序の不定性、rollback困難などの問題があった。

そこで、workerをartifact producerへ限定し、live Notebookへの反映をsingle committerだけに許可する設計へ移行した。

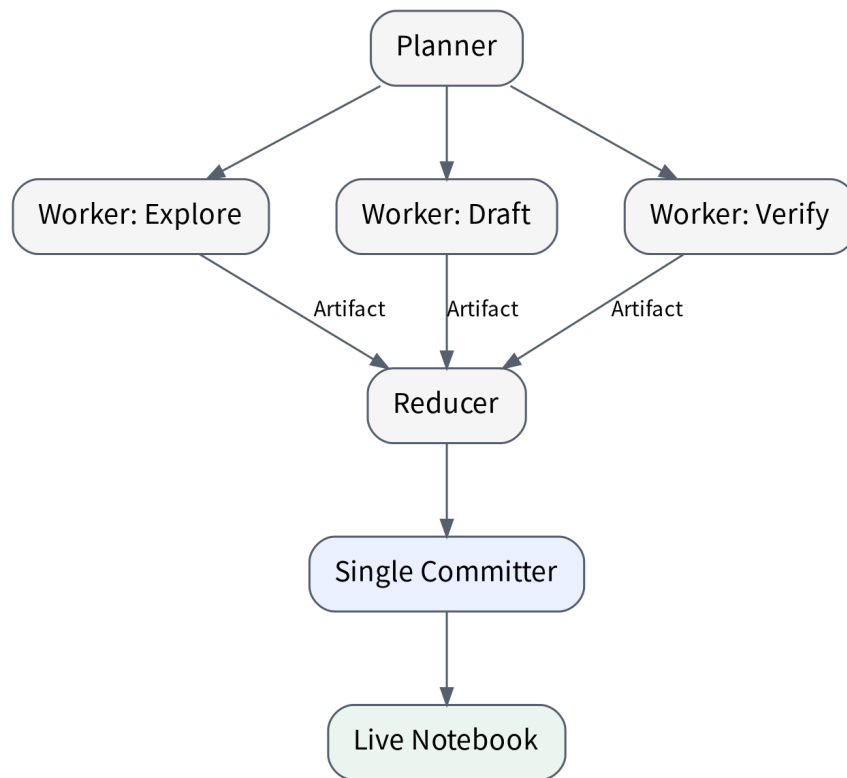


図5 ClaudeOrchestratorのworker/reducer/single-committer構造

worker間で共有されるのは、Notebookそのものではなく、artifact、JSON、Association、明示的なreferenceである。

### 4.3 主要ファイル

| ファイル                                 | 主な役割                                        |
|--------------------------------------|---------------------------------------------|
| ClaudeOrchestrator.wl                | role、capability、planner、spawn、reduce、commit |
| ClaudeOrchestrator_workflow.wl       | multi-token Petri net                       |
| ClaudeOrchestrator_promptworkflow.wl | 複雑promptのworkflow化                          |
| ClaudeOrchestrator_observability.wl  | LLM呼び出し、handler、workflow観測                  |
| ClaudeOrchestrator_session.wl        | session補助                                   |
| ClaudeOrchestrator_info/             | 設計・API・例                                    |

### 4.4 RoleとCapability

代表的roleはExplore、Plan、Draft、Verify、Reduce、Commitである。

各roleにはcapabilityが割り当てられ、worker roleにはNotebookへの直接書き込み権限を与えない。

代表的なdeny対象にはNotebookWrite、CreateNotebook、EvaluationNotebook、RunProcess、SystemCredential、無制限なURLRead、live resourceへの直接副作用などが含まれる。

## 4.5 実行パイプライン — Planning ・ Spawn ・ Reduce ・ Commit

### 4.5.1 Planning

ClaudePlanTasksは親タスクをTaskSpec DAGへ分解する。TaskSpecは次のフィールドを持つ。

```
TaskId
Role
Goal
Inputs
Outputs
Capabilities
DependsOn
ExpectedArtifactType
OutputSchema
```

Plannerの出力はそのまま実行せず、schemaと依存関係が検証される。

### 4.5.2 Spawn

ClaudeSpawnWorkersは依存関係を見ながらworkerを起動し、成果物を収集する。

```
Task A: Explore
Task B: Draft, depends on A
Task C: Verify, depends on B
```

各workerは自分のTaskSpec、許可された入力、capability、出力schemaだけを受け取る。結果はArtifacts: taskId -> artifactとFailuresへ集約される。

実行順序の既定は `MaxParallelism -> 1` の直列実行であり、 `MaxParallelism > 1` または `UseDAG -> True` を指定すると依存関係を保ったDAG並列経路へ切り替わる【実装済】。なお原本間差分として、API文書には「直列／順次、現状1」という古い記述が残っている（付録C参照）。

### 4.5.3 Reduce

ClaudeReduceArtifactsは複数artifactを統合する。Reductionは決定的処理またはLLMによる統合を利用できるが、source reference、task reference、検証状態を保持しなければならない。

Reducerはlive Notebookへ書き込まず、ReducedArtifactを生成する。

### 4.5.4 Commit

ClaudeCommitArtifactsがsingle committerとして最終反映を行う。commit経路は `CommitMode` オプションで選択され、既定は `"Direct"`（承認済みpathを通した直接反映）である。次の図はshadow buffer・検証・rollbackを含む `CommitMode -> "Transactional"` を選んだ場合の経路である。



```
Reduced Artifact
↓
Schema validation
↓
Shadow buffer
↓
Verifier
↓
Commit plan
↓
NotebookWrite through approved path
↓
Post-commit verification
↓
Rollback if required
```

Transactional modeでは、workerの誤出力が直接Notebookを壊すことを防ぎ、失敗時にはtargetを無変更にて保てる。既定のDirect modeにはこのshadow buffer／rollback段がないため、commitの事故耐性を評価する際はどちらのmodeかを区別する必要がある。

## 4.6 Petri net workflow

ClaudeOrchestrator\_workflow.wlは、単純なDAGより一般的なmulti-token Petri netを提供する。Petri netは、複数条件が揃った時だけ遷移、複数token、retry、approval待ち、外部job完了待ち、branch join、cancellation、timeout、repair loopなどを表現しやすい。

## 4.7 PromptWorkflow — 複雑な自然言語要求をworkflow候補へ変換する

ClaudeOrchestrator\_promptworkflow.wlは、単一関数routeで表現しにくい複雑なpromptをWorkflowNet候補へ変換する層である。一方、短い事実質問や単純な関数照会がRuntimeのsingle pathへ戻す分類も持つ。

ここでも新規workflowをLLMが出力した瞬間に実行することはない。まず既存のFunctionRoute/WorkflowRouteが一意に一致すればそれを優先し、新規生成を避ける。既存routeがなければ、LLMを呼ぶ前にローカルな複雑度判定を行う。明示的な「workflow化」要求、複数のsubtask動詞、あるいはtask動詞と「まず/次に/finally」等の制御語が共存する場合にWorkflowCandidateとなる。

候補となった場合、ClaudeProposeWorkflowNetFromPromptがWorkflowNetコードを提案するが、禁止パターンの静的走査、held parse、宣言的構造の抽出を経る。新しく生成されたworkflowは、promptに「作ってすぐ実行」と書かれていてもWorkflowRouteDraftのNeedsApprovalで停止し、自動登録・自動実行しない。これは「LLMがコードを生成する能力」と「workflowをauthorityとして有効化する権限」を分ける重要な境界である。

図7 ClaudeEval・PromptRouter・PromptWorkflow・仕様実装ワークフローの関係

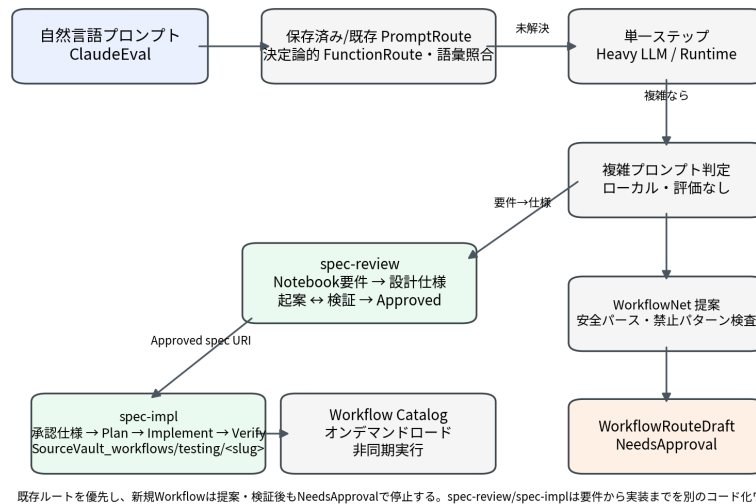


図6 ClaudeEval・PromptRouter・PromptWorkflow・仕様実装workflowの関係

## 4.8 要件から仕様生成、さらに実装までをworkflow化する

SourceVault\_workflowsには、PromptWorkflowとは別に、ソフトウェア開発自体を二段階で自動化するsystem workflowとしてspec-reviewとspec-implが存在する。spec-reviewはNotebook上の要件テキストを入力とし、起案モデル（`$ClaudeModel1`）と検証モデル（`$ClaudeAdvisaryModel1`、既定Codex）の往復によってWolfram設計仕様MarkdownをApprovedまで収束させる。承認された仕様とreviewはSourceVault immutable snapshotとして保存され、承認仕様はsv://snapshot/OrchSpec/...で参照できる。

spec-implは承認済み仕様を受け取り、Plan → optional plan review → Implement → Verifyを反復し、`SourceVault_workflows/testing/<slug>/`にロード可能な`SVWorkflow_<Canon>`パッケージを生成する。検証はLLM reviewだけに依存せず、静的smoke、fresh wolframscriptでの動的load+launchというhard gate、生成testの実行、最後に仕様適合のLLM検証という複数段階を持つ。新規生成物は必ずtesting stageに入り、テスト後にproductionへ昇格する。

| 段階          | 入力                   | 主な処理                            | SourceVault上の成果物                                                                                          |
|-------------|----------------------|---------------------------------|-----------------------------------------------------------------------------------------------------------|
| spec-review | Notebook要件           | 起案 ↔ 検証をApprovedまで反復            | <code>orch/&lt;project&gt;/{requirements,spec,review}</code> ;<br><code>sv://snapshot/OrchSpec/...</code> |
| spec-impl   | 承認済み設計仕様             | Plan → Implement → Verify       | <code>impl/&lt;slug&gt;/{plan,planreview,artifact,verify}</code>                                          |
| Catalog登録   | 検証済みworkflow package | testing stageへ生成、on-demand load | <code>SourceVault_workflows/testing/&lt;slug&gt;/</code>                                                  |

重いLLMループはFront End kernelを固めないよう背景wolframscriptで実行し、FE側は共有polling tickでprogress/resultを監視する。失敗時も途中のplan、artifact、verify chainをSourceVaultへ残すため、単なるコード生成ではなく、設計過程と検証過程を再構成できる点がSourceVaultらしい特徴である。

主 な コ ー ド ・ 文 書 根 拠 : `ClaudeOrchestrator_promptworkflow.wl` / `SourceVault_workflows/spec-review/SVWorkflow_SpecReview_info/design/implementation-spec.md` / `SourceVault_workflows/spec-impl/SVWorkflow_SpecImpl_info/design/implementation-spec.md` / `SourceVault_info/docs/api_workflowregistry.md`

## 4.9 評価

ClaudeOrchestratorの最も強い設計判断は、workerとcommitterの分離である。一般的なmulti-agent systemでは、各agentがtoolを直接呼び、副作用を共有環境へ与える場合がある。ここではworkerをartifact producerへ限定し、commitを一つの責務として明示したことで、Notebook整合性と監査性を高めている。

## 4.10 今後の展望

今後はRuntimeSessionとPetri netの境界を明確にすることが重要である。各model turnや通常tool callをPetri tokenへ展開すると状態爆発が起きるため、Orchestratorはepisodeの開始・approval・budget・artifact commitだけを管理し、内部のturn/tool loopはRuntimeへ閉じ込める方向が適している。

agent間の共有については、SourceVaultをartifact busとして使う設計がさらに重要になる【部分実装／統合目標】。worker AがMCP depositしたsv://artifact/...をworker Bが検索・取得し、Reducerが参照関係を維持したまま統合する構成なら、Notebookを共有メモリとして乱用せずに済む。deposit／search／getの各APIは実装済みだが、全worker間通信がこの経路へ統一されているわけではない。

長期的には、workflow catalog、AutoTrigger、multi-PC placementを統合し、同じworkflowを手動、PromptRouter、schedule、SourceVault event、remote PCから一貫して起動できるようにすることが目標になる。その際、workflow自動生成と自動実行の間にはapproval・stage promotion・diagnostics gateを残すべきである。

## 第5章 SourceVaultの基礎アーキテクチャ

SourceVaultは、単なる文書検索データベースではなく、「何を根拠に、どの版を使い、何を生成したか」を追跡するsource-first knowledge vaultとして設計されている。URLやPDF、arXiv、Notebook、メールをsourceとして登録し、その時点の内容をimmutable snapshotとして保存し、要約・claim・検索chunk・artifactを派生物として分離する。この設計の狙いは、上流資料が変化したときに影響範囲を追えることにある。URLは同じでも内容が更新されれば別snapshotになり、古いsnapshotを根拠にしたEvidence Bundleや生成物をstale候補として扱える。したがってSourceVaultの中心価値は検索速度よりも、source identity、version、provenance、再実行可能性にある。

外部との交換ではsv:// URIが重要になる。object、snapshot、artifact、chunk等を安定したopaque referenceとして表現し、MCPやworkflowを跨いで同じ対象を指せるようにする。これにより、LLMへ巨大な本文を直接渡すのではなく、必要な時に必要なprojectionをrelease gate付きで取得する構造を作れる。このURI層を外部LLMクライアントへ開く窓口がSourceVault MCPであり、authorityではなくprojection layerとして設計されている点に意義がある。

以下、5.1～5.4でsource-first設計とlifecycle管理を、5.5～5.6で決定論的利用経路と検索のrelease gateを、5.7～5.9でsv:// URI・MCP・外部LLM連携を述べ、5.10で評価、5.11で今後の展望をまとめる。

### 5.1 Source-First設計 — Source・Snapshot・Derived Artifact

SourceVaultは四リポジトリの中で最も大きく、機能範囲も広い。基本思想は、LLMが生成した要約や回答より先に、sourceそのものの同一性、版、取得時刻、内容hash、provenanceを正準化することである。対象となるsourceにはURL、Webページ、PDF、arXiv、Notebook、テキスト、メール、Eagle管理資料、生成artifact、workflow resultなどが含まれる。

SourceVaultでは、論理的なsourceと、その時点の内容を表すsnapshotを分離する。

```
Source
├─ Snapshot v1
├─ Snapshot v2
└─ Snapshot v3
```

同じURLや論文でも、時間とともに内容が更新される可能性がある。Sourceは同一性を表し、Snapshotは内容hashに基づくimmutableな版を表す。

要約、claim、embedding、検索index、生成コードは元sourceとは別のderived artifactとして扱う。

```
Snapshot
├─ Parsed Text
├─ Claims
├─ Summary
├─ Search Chunks
└─ Evidence Bundle
```

この分離により、元資料が更新されたときに、どのderived artifactが古くなったかを判定できる。

### 5.2 モジュール構成

| ファイル                     | 主な役割                                   |
|--------------------------|----------------------------------------|
| SourceVault.wl           | umbrella loaderと統合入口                   |
| SourceVault_core.wl      | immutable snapshot、event、vault core    |
| SourceVault_contracts.wl | FunctionContract、failure schema、初期化DAG |

| ファイル                          | 主な役割                                  |
|-------------------------------|---------------------------------------|
| SourceVault_promptrouter.wl   | PromptRouter、保存prompt、privacy routing |
| SourceVault_searchindex.wl    | release contextと検索index               |
| SourceVault_searchview.wl     | 検索結果をたどる作業面                           |
| SourceVault_servicemanager.wl | service、job、heartbeat                 |
| SourceVault_webingest.wl      | Web／PDF取得と解析                          |
| SourceVault_identity.wl       | Identifier／Entity                     |
| SourceVault_mining.wl         | Tag／Authorship／MiningObject           |
| SourceVault_oopsseed.wl       | OOPS seed ontology                    |
| SourceVault_mailstructure.wl  | 一般メールの構造化                             |
| SourceVault_knowledgehome.wl  | Knowledge Home                        |
| SourceVault_diagnostics.wl    | SystemDoctor                          |
| SourceVault_capbroker.wl      | CapabilityLease／PreparedInputToken    |
| SourceVault_taint.wl          | input trustとtaint                     |
| SourceVault_cognition.wl      | 認知支援                                  |
| SourceVault_adjudication.wl   | 複数LLM裁定                               |
| SourceVault_anomaly.wl        | observe-only異常分析                      |

機能数が多いため、SourceVaultは戻り値、failure、初期化依存を契約化する層（SourceVault\_contracts.wlのFunctionContract）を持つ。標準化されたFailure schemaを用いることで、単なる `$Failed` ではなく、原因、公開可能なreason、内部診断を区別できる。

## 5.3 Ingestフロー

```

Input Source
  ↓
Resolve Identity
  ↓
Fetch Raw Bytes
  ↓
Content Hash
  ↓
Immutable Snapshot
  ↓
Parse / Chunk
  ├── Index
  ├── Claim Extraction
  └── Evidence Bundle

```

raw dataは内容hash単位で保存され、metadataはsourceとsnapshotを参照する。同じ内容を別URLから取得した場合にも、content hashを用いて重複を扱える。

## 5.4 ClaimとEvidence Bundle

Claimは、sourceから抽出された主張である。

```

Claim
├─ Claim Text
├─ Source Ref
├─ Snapshot Ref
├─ Span Ref
├─ Confidence
└─ Extractor / Run

```

claimは元sourceを置き換えず、append-onlyな派生記録として管理される。

Evidence Bundleは、成果物が依存するsource、snapshot、claim、generated fileをまとめる。

```

Evidence Bundle
├─ Sources
├─ Snapshots
├─ Claims
├─ Generated Files
└─ Generator / Workflow

```

上流snapshotがStaleまたはInvalidatedになった場合、そのsnapshotを参照するbundleも再評価対象になる。

## 5.5 PromptRouterと決定論的利用経路

PromptRouterは、自然言語promptを、決定的な関数呼び出しまたはworkflow proposalへ変換する。重要なのは、Router自身が式を評価しない点である。

```

Natural Language Prompt
↓
Route Proposal
↓
Held / Unevaluated Expression
↓
NBAccess / Runtime Validation
↓
Execution

```

曖昧な場合には勝手に実行せず、候補またはNeedsApprovalを返す。

利用者の視点から見ると、SourceVaultは「LLMのためのvector store」から始めるのではなく、まずsourceの同一性と版を登録する。基本操作はSourceVaultIngestによるURL/PDF/テキスト/arXivの取り込み、SourceVaultIndexNotebookによるNotebook index、SourceVaultStatus/ListSources/ListSnapshotsによる状態確認である。内容hashが同じ再取り込みは重複を作らず、sourceのstable identityと時点ごとのsnapshotを分離する。

利用経路は決定論的経路とLLM経路に分かれる。ingest、Notebook index、lint、FindNotebooks、既知PromptRouteなどはLLMなしで動作できる。構造化claim抽出やNotebook summaryなど、意味理解が必要な処理はClaudeRuntime経由のLLMを利用する。この分離により、source管理そのものを特定LLM providerへ依存させない。

| 利用場面       | 代表的な操作                                    | LLM依存               |
|------------|-------------------------------------------|---------------------|
| 資料登録       | SourceVaultIngest[URL/PDF/text/arXiv]     | 不要                  |
| Notebook管理 | SourceVaultIndexNotebook、Header/ToDo/Lint | 不要                  |
| 証拠参照       | Span/Context、snapshot、Evidence Bundle     | 不要                  |
| 高レベル照会     | PromptRouterによるUpcomingSchedule等          | route次第。決定論routeは不要 |

| 利用場面    | 代表的な操作                                           | LLM依存                          |
|---------|--------------------------------------------------|--------------------------------|
| 意味抽出    | LLM schemaによるSourceVaultExtract、Notebook summary | ClaudeRuntime経由                |
| 複雑作業    | Orchestrator workflow、spec-review/spec-impl      | 複数LLMを利用可能                     |
| 外部LLM連携 | MCP search/get/directives/deposit                | client/modelのaccess profileに依存 |

## 5.6 Release Contextとfail-closed検索

SourceVault検索では、ReleaseContextを必須とする。gate判定に関与する値は、policy側とデータ側に分けて理解する必要がある。

まず、登録されるReleaseContextのpolicy値は次のとおりである。

```
MaxPrivacyLevel    (必須。返却してよい上限privacy)
RequiredTags       (持っていることを要求するtag。allow-list判定)
DenyTags          (持っていたら拒否するtag)
ReleaseContextTag / Sink / DisplayName / RequireCitation /
AllowAnswerGeneration など (booleanの既定は安全側False)
```

一方、release対象となるsource/chunk側のmetadataとして、PrivacyLevel、Tags、State (Approved / Published / Releasedのみgate通過)、ValidFrom / ValidUntil (有効期限) が保持され、gateはpolicy値とこれら突き合わせて判定する。StateやValidFrom/ValidUntilはReleaseContext自体のフィールドではない点に注意が必要である。

検索結果はbuild-timeだけでなく、返却直前にもrequest-time gateで再評価され、build時のRevocationEpochが失効していれば返却を拒否する。なお、設計仕様はReleaseContext・AccessRequest・RevocationEpochを合成した「GateContext」という概念で判定を記述するが、これは一つの公開API型として実装されているわけではない【設計】。searchindexのrelease policyと、MCP側のAccessRequest／AccessGrant・RevocationEpochは、それぞれ別の公開APIである。

```
Search Candidate
↓
Request-time Release Policy
↓
Revocation Check
↓
Safety Check
↓
Permit only
```

未登録contextや不完全なcontextは、広く許可するのではなくfail-closedで空結果または拒否になる。なお、LLM routingで使うTrustDomain (cloud/localの送信先区分) や、CapabilityLeaseのPurposeは、それぞれPromptRouter・Capability Broker側の概念であり、ReleaseContextのフィールドではない。同種の「用途を絞る」発想が層ごとに独立して実装されている。

## 5.7 sv:// URI — SourceVaultを跨いで使う正準identity

SourceVault Universal MCP Access仕様は、SourceVault/MCP/Web/Orchestrator間で交換する参照をsv://... URIへ正準化する方針を定めている。重要なのは、URIにmail、web、pdfといったdata classを埋め込まない点である。媒体種別はClass/MediaType/Kindというsidecar metadataに置き、URIはopaqueなidentityに徹する。

仕様v2が第1セグメントとして正準予約するnamespaceは、object、chunk、artifact、hash、group、relation、snapshot、record、citationの九つである。現行のURI parser (`$SourceVaultURINamespaces`) はこれに加えてfile、packageapi、directive

の三つを予約しており、計12 namespaceを受理する。ただし「parserで予約済み」「toolの出力として使用中」「data adapterとして結線済み」は別の状態である点に注意が必要である。

| namespace例                                                 | 用途                                             | 位置づけ                                                                                                       |
|------------------------------------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>sv://object/&lt;id&gt;</code>                        | 一般object identity                              | v2正準予約                                                                                                     |
| <code>sv://chunk/&lt;id&gt;</code>                         | 検索・grounding用chunk                             | v2正準予約                                                                                                     |
| <code>sv://artifact/&lt;id&gt;</code>                      | LLMやworkflowが生成したartifact                      | v2正準予約                                                                                                     |
| <code>sv://hash/sha256/&lt;hex&gt;</code>                  | content-addressed hash                         | v2正準予約                                                                                                     |
| <code>sv://snapshot/&lt;class&gt;/&lt;hex&gt;</code>       | immutable snapshot                             | v2正準予約                                                                                                     |
| <code>sv://group/... / relation / citation / record</code> | 関係、group、citation、record                       | v2正準予約                                                                                                     |
| <code>sv://file/&lt;id&gt;</code>                          | allow-listed file reference (Directivesもここへ投影) | 現行parserで予約済み (v2予約外の実装拡張)。adapter結線済み                                                                     |
| <code>sv://packageapi/&lt;id&gt;</code>                    | package API contract/document                  | 現行parserで予約済み (v2予約外の実装拡張)。adapter結線済み                                                                     |
| <code>sv://directive/&lt;id&gt;</code>                     | directiveのstable opaque id                     | 現行parserで予約済み。独立adapterは未結線で、<br><code>sourcevault_directives</code> toolの現行出力は <code>sv://file/...</code> |

つまりdirectiveについては、URI grammar上の予約は実装済みだが、retrieval仕様が定める独立directive adapterへの全面移行は未完了であり、現行のtool出力はfile namespaceを経由する【部分実装】。

SourceVaultCanonicalURIは、URIをkey、edge、group member、SourceRefとして保存する前の正規化点である。従来のblob:sha256:...やsnapshot:class:...は互換入力として受理され、sv://hash/...やsv://snapshot/...へcanonicalizeできる。したがって「SourceVault内の全内部レコードがすでにURIだけで保存される」と言うより、外部API・検索・関係記述における安定した共通参照層をsv://へ統一している、と表現するのが正確である。URIはidentityであってcapabilityではないため、URIを知っていてもread権限は得られない。

## 5.8 SourceVault MCP — projection layerとしての外部LLM窓口

LLMにSourceVaultを使わせる方式は一つではない。PromptDeliveryProfileは、modelと実行環境がMCPを利用できるか、ローカルfolderを読めるか、SourceVault入力が完全にmediatedされているかを見て、MCPDeferred、InlineContext、Hybrid、LocalToolRefs、NoSourceVaultのstrategyを選択する。これにより、Claude Codeのようにローカルtool harnessを持つcloud-backed modelと、LM Studioのlocal modelとを同じものとして扱わず、provider trustとデータreleaseを分離できる。

特にClaude Code/Codexはローカルfilesystemやshellを操作できても、モデル推論自体はcloud/remoteであるため、MCP release判定ではLocalLLMとして自動的に高privacy本文を返してはならない。逆に、信頼されたlocal/private modelには登録済みAccessProfileに応じてより高いaccess ceilingを設定できる。

SourceVault\_mcp.wlはMCP protocol transportそのものではなく、tool schema、initialize/tools/list/tools/call dispatch、argument validation、provenance、JSON-safe responseを担当するWolfram Language側ライブラリである。HTTP/JSON-RPC transportはPython proxyが受け、file command queueを介してservice kernelのSourceVaultMCPDispatchへ渡す。したがってMCPはSourceVault data storeのauthorityではなく、安全なprojection layerである。read/write/cloud-sendの最終判定はrelease policy、AccessGrant、NBAccess等へ委譲される。



図8 MCPを介したSourceVaultのURI参照・Directive共有・Artifact交換

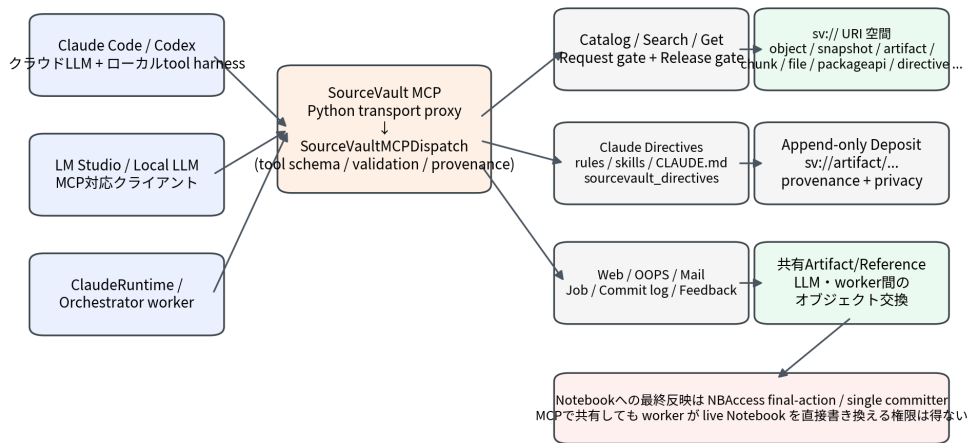


図7 MCPを介したURI参照・Directive共有・Artifact交換

現行SourceVault MCPのtool集合は、単なる検索APIを超えている。以下は機能別に整理した26 toolである。

| 分類                    | tool                                                                                                                                | 役割                                                                   |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Universal core        | sourcevault_catalog / sourcevault_search / sourcevault_get                                                                          | adapter catalog、横断検索、URI単体取得                                         |
| Access                | sourcevault_request_access / sourcevault_access_status                                                                              | body等に必要なAccessGrant申請とpoll                                          |
| Capability            | sourcevault_runtime_capabilities                                                                                                    | model/client別のPromptDeliveryProfileと利用可能能力を返す                        |
| Web                   | sourcevault_web_search / sourcevault_submit_web_search / sourcevault_job_status / sourcevault_job_result / sourcevault_get_document | 同期/非同期Web検索、job、取得文書                                                 |
| Filesystem            | sourcevault_fs_list / sourcevault_fs_read                                                                                           | allow-listされたpackage/working directoryの列挙・読取。secret-looking fileは拒否  |
| Directives            | sourcevault_directives                                                                                                              | Claude Directivesのrules/skills/root instructionを列挙しURIで返す            |
| Package history       | sourcevault_commit_log                                                                                                              | GitHub API由来のpackage commit metadata                                 |
| OOPS                  | sourcevault_oops_status / sourcevault_oops_search_threads / sourcevault_oops_thread                                                 | OOPS corpusのstatus・thread検索・thread取得                                 |
| General mail          | sourcevault_mail_status / sourcevault_mail_search_threads / sourcevault_mail_thread                                                 | 一般メール構造化データの検索                                                       |
| Workflow authoring    | sourcevault_workflow_write                                                                                                          | API/LM Studio系clientがSourceVault_workflows配下へ安全にworkflow fileを書き込む補助 |
| Orchestrator feedback | sourcevault_orchestrator_help / sourcevault_orchestrator_subtask                                                                    | help request・subtask proposalをOrchestrator側へbridge                   |
| Feedback              | sourcevault_feedback_submit                                                                                                         | critique、correction、help等のappend-only feedback envelope              |
| Write                 | sourcevault_deposit                                                                                                                 | LLM生成artifactのappend-only deposit                                    |

sourcevault\_searchはadapterを横断し、SearchSpecを正規化したうえで各resultをrequest-time release gateに通す。

sourcevault\_getはsv:// URIを解決し、既定では低漏洩summary/metadataだけを返す。原則としてbody/raw/contextはAccessGrantを要求し、機密本文のcloud sinkへの送付はgrantがあっても拒否される。つまりMCP clientが検索できることと、

本文を取得できることは分離されている。ただし例外として、adapterが`BodyGrantRequired -> False`を宣言するPublicDoc文書の`view="body"`はgrant不要でrelease gateだけを通る。組み込みpackageapi adapterはこの例外を使い、PrivacyLevel 0のpackage API文書本文をcloud clientへも返却できる。この例外はbodyに限られ、一般のraw/contextをgrant不要にするものではない。

## 5.9 DirectivesのMCP共有とartifact deposit

### 5.9.1 Claude DirectivesをMCPデータベースとして共有する意味

claudecode側のClaude Directivesは、CLAUDE.md、rules、skillsからなる正準directive repositoryであり、Claude CLI向けにはclaude/、Codex向けにはAGENTS.mdや.agents/skills等へ実ファイルとして生成・同期（materialize）できる。Claude CodeやCodexはこのローカルharnessを直接利用できる。一方、SourceVault MCPのsourcevault\_directivesは同じdirective集合を検索・参照可能なデータとして公開し、rule/skillをsv://file等の参照として返す。

これにより、MCP toolを推論中に呼べるLM Studio等のローカルLLMクライアントも、Claude Code/Codexと同じrules・skillsを必要時に参照できる。これはmodelごとに別々のsystem promptを手作業で複製するのではなく、directiveを一つの管理対象としてSourceVault/MCPから投影する点に意味がある。ただし、どのdirectiveをどのmodelへ開示するかはAccessProfileとrelease policyの対象であり、MCP可視性だけで全データが解放されるわけではない。

### 5.9.2 MCPによる書き込み — sourcevault\_deposit

sourcevault\_depositはLLMが生成したmarkdown、Wolfram Language、PDF、image、JSON等をSourceVault管理下へ能動的に保存するappend-only APIである。既存objectの上書き・削除・privacy緩和を直接許すmutationではない。commit時にはDepositArtifact権限を持つAccessGrantまたはmodel AccessProfileを必要とし、provenance（authoredBy、inputRefs、citationRefs、promptRefs）に加え、spec上位のSessionId・BatchIdを記録する。成功するとsv://artifact/...が返り、同一idempotency keyと同一内容はreplayとして扱われる。

privacyはLLM自身の申告だけでなく、そのSession/Batchで実際にreleaseされた入力の最大privacyを継承する。sourceRefsがあるのに観測根拠がない場合はfail-closed floor（privacy 0.75）を適用する。したがって、private sourceを読んだagentが「出力はpublic」と自己申告してprivacy launderingすることを防ぐ方向の設計になっている。

### 5.9.3 SourceVaultをLLM/worker間のオブジェクト共有面として使う

ClaudeOrchestratorの原則ではworkerはlive Notebookへ直接commitできず、成果物はartifactとして境界を越える。一方MCP depositはLLMがartifactをSourceVaultへ保存し、sv://artifact/...というopaque referenceを発行でき、別のLLM/workerはMCP search/getを通じて許可されたprojectionを取得できる。この組み合わせにより、SourceVaultを「Notebookを共有memoryとして直接編集する」代替の、provenance付きオブジェクト交換面として利用できる。

これは現時点で「全worker間通信が自動的にMCP/SV URIへ統一済み」という意味ではない。むしろ、single-committer制約を崩さずにagent間共有を行うための構成原理である。大きなartifact本文をPetri tokenやconversationに埋めずreferenceで渡すOrchestratorのsession/episode設計とも整合する。最終的なNotebookへの反映は、あくまでreducer/committerまたはNBAccessのPendingFinalActionQueueを通す。

```
主なコード・文書根拠: SourceVault_info/docs/user_manual.md / SourceVault_info/docs/api_mcp.md / SourceVault_mcp.wl /
SourceVault_info/design/sourcevault_universal_mcp_access_spec_v2.md / claudecode_info/docs/api_directives.md /
ClaudeOrchestrator_info/design/claude_orchestrator_runtime_session_episode_petri_spec_v0_1.md
```

## 5.10 評価

SourceVaultは基礎アーキテクチャの段階でも単なる文書格納庫ではない。immutable snapshot、claim、Evidence Bundle、Notebook management、PromptRouter、release contextを統合し、source-centricな証拠と決定的再実行を提供している。こ

の基礎があるからこそ、第8章で述べるマイニングレイヤーは「単に似た文章を探す」だけでなく、どのsnapshot、span、person、topic、workflowに由来する関係かを保持できる。

## 5.11 今後の展望

今後のSourceVault coreでは、sv:// URI resolverを全object classへ拡張し、legacy refとのalias解決を安定化することが重要である。URIが検索、MCP、workflow、artifact relation、citationの共通キーになれば、内部IDの違いを外部clientへ露出せずに済む。

また、source lifecycleとderived artifactのstaleness propagationを、より自動的に評価できる余地がある。snapshot更新時に、どのclaim、summary、workflow result、Notebook記述が再検証対象かを依存グラフから計算し、review queueへ出せば、長期知識基盤としての価値が高まる。

MCP depositやagent artifactが増えるほど、自己生成物と外部一次資料を混同しないことも重要になる。ArtifactType、SourceRefs、AuthoredBy、DepositedBy、confidence、review stateを明確に分け、LLM生成物を自動的に根拠へ昇格させない設計を維持する必要がある。

## 第6章 コードレベルの統合技術解析

本章は、四つのリポジトリに共通する設計原則をコードレベルで読み解く章である。個々のパッケージを別々に見ると独立した機能に見えるが、権限の合成方法、提案と実行の分離、状態の持ち方、実行場所の選択には一貫した思想がある。この共通原則を明示することの意義は、今後パッケージが増えても「どこに何を実装すべきか」の判断基準を提供する点にある。

最も重要な原則は「権限は上位から下位へ足すのではなく、下位の基本許可を上位スコープで狭める」という考え方である。NBAccessのbase policyにRuntime scope、Workflow scope、Step scopeを積集合として重ねるため、上位層の便利機能が安全境界を越えて権限を拡張しにくい。第二の原則は、提案と実行、artifactとlive state、immutable recordとcurrent projectionを分けることである。LLMはproposalを返し、workerはartifactを返し、SourceVaultはappend-only eventを残し、現在状態はそこから投影する。これらは別々の機能に見えるが、共通して「途中結果を直接正本にしない」という設計思想を共有している。第三の原則は、実行場所もpolicyの一部として扱うことである。main kernel、subkernel、wolframscript、SpecificMachine、remote PCは性能だけでなく、FrontEnd依存性、secret access、license seat、fault isolationによって使い分けられる。

以下、6.1～6.5で各原則の実装形を、6.6で責務境界の一覧を示し、6.7で今後の展望をまとめる。

### 6.1 下位境界へ安全性を集約する設計

四リポジトリをコードレベルで見ると、上位層が独自に安全性を再実装するのではなく、下位境界へ判断を委譲する構造が一貫している。

```
SourceVault / Orchestrator / Runtime Scope
      ↓ intersect
    NBAccess Base Policy
      ↓
    Final Decision
```

最終権限は概念的に次の積集合となる。

```
FinalPermission
= NBAccessBase
n RuntimeScope
n WorkflowScope
n StepScope
```

ただしこれは【概念モデル】であり、この名前の共通policy合成関数が一つ実装されているわけではない。実際には各層が異なる語彙で同じ原則を実装している。NBAccessはPrivacySpec / AccessSpec、OrchestratorはTaskSpecのCapabilities、SourceVaultはReleaseContextとCapabilityLeaseである。共通するのは、RuntimeやOrchestratorが権限を狭めることはできても、NBAccessの基本許可を越えて広げることはいけないという方向性であり、これらを単一のpolicy digest / contractへ統合することは将来課題である（6.7参照）。

### 6.2 提案と実行の分離

LLM出力はproposalであり、命令ではない。

```
LLM
↓
Proposal
↓
Parse
↓
Hold
↓
Validate
↓
Decision
↓
Execute
```

この原則はRuntimeだけでなく、PromptRouter、Orchestrator planner、multi-model adjudicationにも共通している。

## 6.3 Artifactとlive stateの分離

workerはlive Notebookを変更せず、artifactを返す。

```
Worker Result
= Payload
+ Sources
+ TaskRef
+ Schema
+ Validation State
```

live stateへのcommitは別段階で行う。この構造はSourceVaultのderived artifactモデルとも整合する。

## 6.4 Immutableとappend-only

SourceVaultのraw snapshot、claim event、Knowledge Home追記、TagAssertion、AuthorshipAssertion、workflow eventなどは、可能な限りimmutableまたはappend-onlyで管理される。

訂正時に古い記録を削除するのではなく、Superseded、Retracted、decision eventなどを追加する。

```
Original Record
↓
Superseding Event
↓
Current Projection
```

event-sourcedとして実装された各projectionは、event列から再生成できる。ただしこの性質はSourceVault全体の現在状態に無条件で当てはまるわけではない。registryの永続ファイル（例: search profileは `PrivateVault/config/searchindex-profiles.wxf` へ保存・復元される）、machine-localなconfig、access statusのsidecar、cache／pointerなどは、event logとは別の永続化契約を持つ。

## 6.5 Task placementとresource place

| 種類          | 主な条件                                  |
|-------------|---------------------------------------|
| Main Kernel | FrontEnd、Notebook、UI、session stateが必要 |

| 種類            | 主な条件                           |
|---------------|--------------------------------|
| Subkernel     | 純粋計算、並列可能、Notebook副作用なし        |
| WolframScript | 長時間、batch、大量I/O、独立processが望ましい |

Runtimeは推薦を行い、NBAccessがactionを検証し、Orchestratorがworkflow上のresource placeとして実行を許可する。

## 6.6 四リポジトリ間の責務境界

| 問題               | 正準担当                                |
|------------------|-------------------------------------|
| Notebookセルの機密性   | NBAccess                            |
| 保持式の実行可否         | NBAccess                            |
| 単一turnの進行        | ClaudeRuntime                       |
| task placement推薦 | ClaudeRuntime                       |
| 複数taskの依存関係      | ClaudeOrchestrator                  |
| worker権限         | ClaudeOrchestrator + NBAccess       |
| Notebook commit  | ClaudeOrchestrator single committer |
| sourceとsnapshot  | SourceVault                         |
| claimとevidence   | SourceVault                         |
| 検索release        | SourceVault                         |
| identity         | SourceVault_identity                |
| 長期topic graph    | SourceVault mining／Knowledge Home   |

この境界が、後半の拡張を理解する前提となる。

## 6.7 今後の展望

今後は、この共通原則をコード規約ではなく機械検証可能なcontractへ昇格させることが望ましい。たとえば「上位層はNBAccess権限を拡張しない」「workerはNotebookWriteを持たない」「source-derived artifactはprovenance refを必須とする」といった不変条件をcontract testとして実行できれば、パッケージ増加による設計崩れを防ぎやすい。

また、run／step単位の共通IDを全層へ通し、PromptRun、Runtime event、Workflow run、MCP call、CapabilityLease、SourceVault artifactを同一traceに結合することが課題である。これにより、障害解析だけでなく、「このNotebook出力はどのLLM入力・どのretrieval span・どのworkflow workerに由来するか」を一貫して説明できる。

multi-PC化が進むと、single-writer、atomic claim、idempotency keyといったローカルで有効だった原則を共有vault上でも維持する必要がある。Dropbox等の同期ストレージは分散DBではないため、per-machine logとmerge viewを基本にする現在の方針をさらに体系化する必要がある。

## 第7章 四層スタックから知識・実行オペレーティングモデルへの発展

前章までに述べた四層は、それだけでも「機密NotebookをLLMと安全に接続し、複雑な作業を複数workerで進め、参照したsourceと生成物を再現可能な形で保存するシステム」として完結している。しかしSourceVaultにURI、MCP、マイニング、identity、AutoTrigger、diagnostics、cognitionが加わると、各機能が循環し始める。取得した知識がworkflowを起動し、workflow結果がartifactとしてSourceVaultへ戻り、そのartifactが次のLLMやagentから参照される。本章の目的は、この循環構造と、それを支える自律実行・複数PC協調の機構を、実装済みの部分と設計段階の部分とを区別しながら示すことである。

この循環を成立させる鍵は、sv:// URIとSourceVaultを共有面として使うことにある。LLMやworkerはNotebookへ直接書かず、artifactをSourceVaultへdepositし、別workerはURIを介して取得する。最終的なNotebook反映はsingle committerが行うため、自律性を上げてもlive stateの統制を失わない。AutoTriggerとSystemDoctorは、この循環を「人がボタンを押す時だけ動くシステム」から「状態変化やscheduleに応じて自律的に動くシステム」へ変える。ただし自律実行の前にhealth gateを置き、診断失敗時は一般workflowを止めるという構造が重要である。

重要なのは、この発展が既存四層を置き換えるものではなく、前半で確認した責務分離があるからこそ成立する点である。マイニングレイヤーが資料間の関係を大量に生成しても、閲覧・送信の可否はrelease gateが判断する。LLMが近傍資料を使って式を提案しても、実行可否はNBAccessが判断する。複数workerが成果物を生成しても、Notebookへの反映はsingle committerが行う。

以下、7.1で閉ループの全体像を、7.2～7.3で自律起動とhealth gateを、7.4～7.5で複数PC協調の現状を述べ、7.6で今後の展望をまとめる。

### 7.1 Prompt・Workflow・MCP・SourceVaultが閉ループを形成する

前章までの機構を接続すると、対話的LLM支援は次の閉ループへ拡張される。ユーザーまたは外部LLMがSourceVaultからsv:// URIでevidenceを取得し、PromptRouterまたはworkflowで処理し、成果物をartifactとしてSourceVaultへ戻す。そのartifactを別workerが参照し、検証後にsingle committerがNotebookへ反映する。さらに処理自体をTriggerSpecとして登録すれば、時刻やSourceVault eventを契機に次回実行を自動化できる。

```
SourceVault source / sv:// refs
  ↓ search・MCP・PromptRouter
Runtime / Orchestrator workflow
  ↓
artifact deposit / snapshot / evidence
  ↓
AutoTriggerがschedule・eventを監視
  ↓
次のworkflow run
  ↓
結果を再びSourceVaultへ
```

この循環が成立することで、SourceVaultは静的なvaultから、LLM agentが読み書きし、workflowが継続的に更新する共有作業面へ性格を変える。ただし自律性はauthorityの無制限化ではなく、readはrelease gate、writeはappend-only deposit/approved mutation、Notebookはsingle committer、external actionはcapability/approvalという境界の内側で実現される。

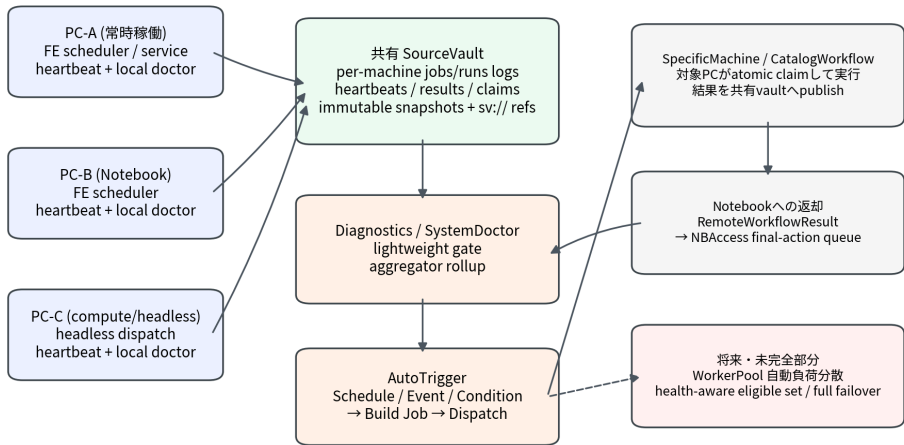
### 7.2 SourceVault\_autotrigger — workflowを安全に自動起動する

SourceVault\_autotriggerはcronを巨大化させるのではなく、Trigger Registry、Scheduler/Watcher、append-only Job Queue、Executorを分離する。TriggerSpecはSchedule、Condition、Target、ExecutionPlacement、DiagnosticsPolicy、RunPolicyを持ち、保存されただけでは自動実行されず、EnabledとAutoRunCapability等の条件を満たす必要がある。



現行 phase1.4 では、trigger の検証・登録、schedule/condition match、diagnostics gate、tick による job 構築、subkernel/Workflow/CatalogWorkflow の dispatch、shared polling tick への scheduler 登録、SpecificMachine placement、remote result 回収まで実装が進んでいる。PromptRoute は main-kernel gated であり、自動 tick からは原則自動実行しない。WorkflowRoute/WorkflowTemplate や CatalogWorkflow は条件を満たせば非同期起動できる。

図9 AutoTrigger・SystemDoctor・共有SourceVaultによる複数PC協調



実装済みのSpecificMachine実行・per-machine single-writer log・atomic claimと、仕様上のWorkerPool/failoverを区別して示す。

図8 AutoTrigger・SystemDoctor・共有SourceVaultによる複数PC協調

### 7.3 最初の自動workflowをSystemDoctorにする設計

AutoTrigger仕様で重要なのは、一般業務workflowより先に包括的なSystemDoctor workflowを自動実行する方針である。仕様 v0.1-r9 では、Diagnostics/SIEMを自動実行の前提層とし、「最初に自動起動されるworkflowはuser workflowではなくSourceVaultSystemDoctor/SourceVault\_diagnosticsを使うsystem doctor workflow」と定義している。一般workflowのEnabled=Trueは、必要componentの直近healthがOKまたは明示的に許容されたDegradedであることを前提とする。

ただし、現行実装との区別が必要である。SourceVault\_diagnosticsはcross-package collector/store/doctor、license probe、service health、machine heartbeat、aggregator rollup等を実装し、AutoTriggerはSourceVaultDiagnosticsLightweightDoctorをdispatch前のcomponent-scoped gateとして利用している。一方、api\_diagnostics.mdはcomprehensive-doctor自動triggerを延期項目として残している。したがって「SystemDoctor first」は設計上の正準方針であり、現時点ではlightweight doctor gateが先行実装された状態と評価する。

### 7.4 複数PCでSourceVaultを共有する

共有SourceVaultを複数PCで運用すると、SourceVault自体がdurableな共通記録面になる。Diagnosticsは各PCが自分専用pathへheartbeatを書き、Aggregatorは全machineのhealthをrollupする。AutoTriggerのjobs/runs logも、単一共有JSONLへ全PCが同時追記する方式から、autotrigger/jobs/<machineTag>.jsonl、autotrigger/runs/<machineTag>.jsonl という per-machine single-writerへ変更されている。これはDropbox等で競合コピーが生じ、リモート追記が消失した実測問題への対策である。

SpecificMachineのCatalogWorkflowは、依頼元PCが共有job logへ「このslugをこのmachineTagで実行」と投入し、対象PCのFE schedulerまたはheadless dispatchが拾う。実行前にatomic claim (autotrigger/claims/<slot>/ のアトミック作成)を行って二重実行を防ぎ、ライセンスprocess slotを確認し、完了後はoutput.wxfを共有vaultへpublishする。依頼元Notebookへの通知はNBAccessのPendingFinalActionQueueを経由するため、リモート実行でもsingle-committer原則を破らない。

| 機能                    | 現行状態 | 冗長化・負荷分散への意味      |
|-----------------------|------|-------------------|
| per-machine heartbeat | 実装済み | 各PCのlivenessを独立観測 |



| 機能                        | 現行状態    | 冗長化・負荷分散への意味                            |
|---------------------------|---------|-----------------------------------------|
| aggregator rollup         | 実装済み    | global healthとactive aggregator候補       |
| per-machine jobs/runs     | 実装済み    | 共有ストレージでの同時追記競合を回避                      |
| SpecificMachine dispatch  | 実装済み    | GPU機・常時稼働機など特定PCへ配置                     |
| atomic dispatch claim     | 実装済み    | FE schedulerとheadless dispatcherの二重実行防止 |
| remote result publication | 実装済み    | 別PCで実行した結果を共有vault経由で取得                 |
| license seat guard        | 実装済み    | 空きprocess枠不足時にdeferし自動再試行               |
| WorkerPool自動負荷分散          | 未実装/設計中 | 複数候補PCから負荷・healthで自動選択する将来層             |
| full failover             | 部分設計    | always-on複数台での縮退・引継ぎは今後の統合対象            |

## 7.5 冗長化・負荷分散の到達点

以上を組み合わせると、すでに「A PCでworkflowを依頼し、B PCで実行し、SourceVaultへ結果をpublishし、A PCのNotebookから取得する」経路は実装されている。またheadless compute nodeもopt-in dispatchで参加できる。これは静的なファイル同期ではなく、diagnostics、placement、claim、result publicationを含む分散実行の基礎である。

一方でEnvironmentIndependent/WorkerPool jobをhealth・負荷・利用可能model/GPU・license capacityに基づいて最適PCへ自動配置し、active node停止時に別nodeへ完全failoverする機能は未完成である。AutoTriggerのAPI文書自身もWorkerPoolの負荷分散・failoverを未実装と明記している。したがって本レポートでは「複数PCの冗長化・負荷分散を実現するための基盤はかなり実装され、SpecificMachine分散は実用段階にあるが、汎用WorkerPool schedulerは将来機能」と評価する。

主なコード・文書根拠: SourceVault\_info/design/sourcevault\_auto\_trigger\_scheduler\_spec\_v0\_1.md / SourceVault\_autotriggger.wl / SourceVault\_info/docs/api\_autotriggger.md / SourceVault\_info/docs/api\_diagnostics.md / SourceVault\_info/docs/user\_manual.md

## 7.6 今後の展望

今後の発展方向は、SourceVaultを中心とするevent-driven knowledge execution fabricの完成である。SourceVault event、schedule、approval state、queue stateを条件としてworkflowを起動し、結果を再びsv://artifactとして保存することで、人手の介在点を必要なapproval境界だけに限定できる。

最初に自動起動するsystem workflowとしてcomprehensive SystemDoctorを位置づけ、その結果に基づいて他workflowの実行可否を決める構成は、運用上分かりやすい。ただし現状はlightweight doctor gateが先行しており、包括doctorの自動triggerや全producer probeのfan-outは発展途上であるため、実装済みと将来像を区別して進める必要がある。

複数PCでは、SpecificMachine実行に加えてWorkerPool、health-aware eligible set、load balancing、failoverを導入することで、共有SourceVaultを小規模な分散実行基盤へ発展させられる。ただし共有ファイル同期の特性上、分散lockを過信せず、per-machine single-writerとatomic claimを正準にする必要がある。

## 第8章 SourceVaultマイニングレイヤー

SourceVaultのマイニングレイヤーは、蓄積された資料を単に全文検索するのではなく、資料同士の関係を発見し、近傍として探索できる知識空間へ変える部分である。通常のベクトル検索では似た文書を近くに置くことが中心になるが、SourceVaultではtopic、author、sender、quote、reply、explicit link、derivation、retrieval historyなど、説明可能な関係を組み合わせる。この設計の狙いは、「なぜこの資料が候補なのか」を根拠付きで提示できる探索空間を作ることにより、opaqueなembedding近傍とは性格が異なる。

この知識空間の初期座標として重要なのがOOPSコーパスである。約6,500通のメールと約4,100件のtopic item、人手付与されたmail-topic対応、topic relation、quote relationがあるため、単に過去メールを検索するだけでなく、新しい一般メールやNotebook、PDF、Webを既存topic空間へ位置づけるseedとして使える。利用者側の見え方がNeighbor Hoppingである。現在読んでいる資料をqueryに変換し、共有topic、直接quote/reply、relation-expanded topicという三つの近傍から次の候補を提示する。検索語を思い出せなくても、資料から資料へ移動しながら知識を掘れる。

以下、8.1～8.2でマイニングの定義と検索の多段構造を、8.3で近傍検索の機構を、8.4で媒体横断投影とidentityを、8.5でOOPSコーパスの構造を、8.6～8.7で一般メールへの拡張とprivacy境界を述べ、8.8で実装状況、8.9で今後の展望をまとめる。

### 8.1 マイニングの定義

SourceVaultにおけるマイニングとは、sourceを単に検索対象へ追加する処理ではない。メール、ローカルファイル、Web、arXiv、Eagle、Notebook、PDF、生成artifactなどのオブジェクト間に、topic、tag、著者・送信者・作成者、人物・組織、引用・被引用、返信・session、明示リンク、派生元・生成先、workflow入力・出力、過去の検索利用履歴、safetyとprivacy、errorとreview状態などの関係を発見・提案・確認する処理である。

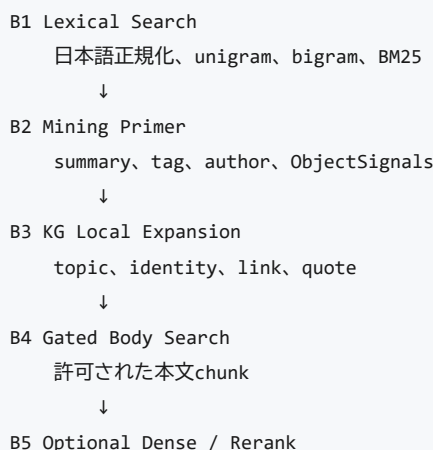
マイニング結果はMiningObjectとして、Confidence、TargetRefs、InputRefs、GeneratedByRunID、ReviewStateなどを持つ。重要なのは、マイニング結果を元資料と区別することである。

```
Original Object
↓
Mining Assertion / Proposal
↓
Human Review or Deterministic Validation
↓
Search Projection
```

LLMが「同一人物らしい」「同じtopicらしい」と推定しても、それを即座に正準事実へしない。

### 8.2 検索の多段構造

SourceVaultの検索は、概念的に次の段階を持つ。



summaryやマイニング結果は、最終回答の根拠ではなく、候補を絞るprimerとして使う。最終的な回答は、原文chunkまたはbody groundingを含まなければならない。

## 8.3 近傍検索とNeighbor Hopping

### 8.3.1 TopicPositionと根拠付きweight

通常の検索では、ユーザーが検索語を入力する。近傍検索では、現在見ているオブジェクトそのものを次の検索の起点にする。まず対象オブジェクトをtopic空間上へ配置する。たとえば、あるPDFのTopicPositionが次のようになっているとする。

| 現在のPDF   |      | 別のPDF    |      |
|----------|------|----------|------|
| セルオートマトン | 0.72 | 可逆計算     | 0.65 |
| 可逆計算     | 0.48 | セルオートマトン | 0.59 |
| 情報熱力学    | 0.31 | Fredkin  | 0.25 |
| 研究会      | 0.18 | 論理回路     | 0.21 |

両者はセルオートマトンと可逆計算を共有するため、近傍候補となる。

ただし、TopicPositionは単なるopaque embeddingではない。各topic weightには根拠（Method）が付く。

| Method           | 意味                 |
|------------------|--------------------|
| OwnerConfirmed   | ownerが明示確認         |
| ExplicitOOPS     | OOPS本文の明示marker    |
| SeedMatched      | seed辞書の表層一致        |
| RelationExpanded | relation graphから展開 |
| AutoExtracted    | 新規topic候補          |
| LLMProposed      | LLMが提案、低いreview状態  |

既存topicで説明できない段落の割合はUnknownMassとして残す。すべてを既存カテゴリへ強制分類しないためである。

### 8.3.2 三種類の近傍

近傍は次の三種類に整理される。

第一は話題上の近傍である。同じtopic itemを共有するオブジェクトであり、媒体がメール、PDF、Notebook、Webのいずれであっても、topic投影が共通なら接続できる。

第二は構造上の近傍である。Reply、Quote、QuotedBy、Attachment、Link、Citation、GeneratedFrom、Supersedes、WorkflowInputなどの直接的な関係を持つオブジェクトである。

第三はrelation展開による近傍である。中心topicからtopic relation graphをたどり、関連topicを担うオブジェクトを探す。

```
セルオートマトン
├─ 可逆計算
├─ 自己増殖
├─ 人工生命
└─ Reservoir Computing
```

展開は無制限に行わず、ホップ数、ノード数、edge weight、per-node top-kで制限する。OOPS auto-tagでは主として重み付き1-hop、検索用KG展開ではcycle-safeなmulti-hop BFSが実装されている。

### 8.3.3 Neighbor Hoppingと3リング提案

SourceVaultKnowledgeHomeNeighborhoodSearchは、対象オブジェクトのTopicPositionを求め、topicが重なるメールやKnowledge Homeパラグラフを返す。

```
現在のメール
↓
同じtopicのメール
↓
そのメールが引用する別メール
↓
関連topicのKnowledge Home追記
```

現在の資料から隣へ移り、さらにその資料を新しい起点として探索する操作がNeighbor Hoppingである。

Knowledge Homeでは、次に読む候補を三つのリングへ分ける。

```
Ring 3: relation-expanded neighborhood
Ring 2: direct quote / reply / link
Ring 1: shared central topics
Current Object
```

Ring 1は中心topicを共有する資料、Ring 2は引用・被引用・返信・明示リンクなどの直接近傍、Ring 3は中心topicからrelation graphを展開した先のtopicを担う資料である。各候補にはReasonsが付与され、なぜ提案されたかを確認できる。

## 8.4 媒体横断の共通投影とidentity

### 8.4.1 共通projection

SourceVaultは媒体の違いを消すのではなく、各原本を保持したまま共通projectionを作る。

```

ObjectURI
ObjectClass
Title
Body / Chunk
Summary
Tags
TopicRefs
Authors
Links
PrivacyLevel
SafetyState
SourceEpoch
DerivedFrom

```

| 媒体       | 主な単位                         | 構造関係                   | identity観測         |
|----------|------------------------------|------------------------|--------------------|
| メール      | Mail、Paragraph、Quote、Session | Reply、Quote、Attachment | From、To、Cc         |
| Notebook | Notebook、Section、Cell、Output | 順序、評価、生成               | Creator            |
| PDF      | Document、Page、Section、Figure | Citation、Version       | Author、XMP Creator |
| Web      | Page、DOM Block、Article       | URL、Link、Snapshot      | schema.org Author  |
| arXiv    | Paper、Version、Reference      | DOI、Citation           | arXiv Author、ORCID |
| Artifact | Output、Run Result            | GeneratedFrom          | LLM、Agent、Workflow |

これにより、たとえばNotebookセルを起点に、同じtopicのPDF、著者が同じWebページ、引用元となったメールをたどる構造を作れる。

#### 8.4.2 SourceVault\_identity — IdentifierとEntity

人物同定では、同一人物が複数の表記を持つ。

```

imai@example.jp
Katsunobu Imai
今井克暢
K. Imai
GitHub: transreal
Notebook Creator: imai_

```

SourceVault\_identity は、Identifier と Entity の二層に分ける。Identifier は Email、URI、SNS、PersonName、OrganizationName、ORCID、arXivAuthor、DOIName、WebAuthorProfile、DocumentCreator、LLMOrAgent など、観測された生の識別情報である。Entity は複数Identifier を束ねる Person、Organization、Bot、Service、MailingList などの実体である。

メールの From、To、Cc は取り込み時に Identifier として登録される。同一 kind と正規化値から決定的 ID が生成されるため、同じメールアドレスを繰り返し取り込んでも同じ Identifier へ upsert される。

#### 8.4.3 AuthorshipAssertion と EntityLinkProposal

媒体と人物の関係は共通 assertion へ投影される。

|          |           |                       |
|----------|-----------|-----------------------|
| Mail     | — Sender  | — Identifier / Entity |
| PDF      | — Author  | — Identifier / Entity |
| Notebook | — Creator | — Identifier / Entity |
| Web      | — Author  | — Identifier / Entity |
| Artifact | — Agent   | — Identifier / Entity |

同一人物が確定していない場合は、Identifierへ直接EntityRefを設定せず、EntityLinkProposalとして保存する。提案にはscore、feature、provenance、review stateを持たせ、人間のaccept/rejectは後の自動再スコアで勝手に覆さない。

identity resolutionはsecurity boundaryではない。既知の人物らしいという理由で、privacy levelを下げたりcloud releaseを許可したりしない。

## 8.5 OOPSコーパス — 知識空間の初期基準座標

### 8.5.1 コーパスとseed index

OOPSは、1992年から2005年の約6,500通のメールと約4,100件のtopic itemを持つコーパスである。SourceVaultでは、これを単なる追加メールデータではなく、Knowledge Homeの初期基準座標として利用する。

主なデータは次のとおりである。

| データ                    | 役割                                                 |
|------------------------|----------------------------------------------------|
| item-name.index        | topic item名、別名                                     |
| mail-to-item.index     | メールからtopicへの対応                                     |
| item-to-mail.index     | topicからメールへの対応（search foundation仕様で規定。parser実装は今後） |
| item-relation.index    | topic間の重み付き有向関係                                    |
| item-relation-up.index | 逆方向のtopic関係                                        |
| quote-table.index      | 引用元メール                                             |
| quoted-table.index     | 被引用関係                                              |
| mail-info.index        | mail番号、list、author                                 |
| oops*.txt              | メール本文                                              |

mail-to-item.indexは人手付与されたgold dataであり、topic推定や検索評価の基準にできる。

### 8.5.2 Owner-scoped topic namespace

OOPSのtopicはグローバルkeywordではない。

```
[ki 2573]
[mi 2573]
[aga 2573]
```

は、それぞれ別namespaceのtopicである。初期構成ではkiをcurrent ownerへ対応させ、mi、agaなどは別ownerのnamespaceとして保持する。同じラベルでもownerが異なれば自動mergeしない。

```
TopicItemRef
Namespace
LocalId
OwnerRef
NamespaceKind
CanonicalLabel
SurfaceForms
PrivacyLevel
SourceRefs
ReviewState
```

未知namespaceも破棄せず、Missing["UnknownOwner"]として残す。

### 8.5.3 メール内部構造とtopic付与

OOPSモデルでは、メールを一つの長い文字列として扱わない。

```
Mail
├─ Header
├─ Prose Paragraph
│   └─ Primary Topic
│       └─ Secondary Topic
├─ Quote Paragraph
│   └─ QuotedFrom
├─ Signature
└─ Footer
```

topic付与の優先順位はExplicitOOPS、SeedMatched、RelationExpanded、AutoExtracted、LLMProposedである。

◎、○、・、[namespace id]などの明示markerは、最高品質の人手付与情報として扱う。seed辞書と本文が一致すればSeedMatchedとなり、そのtopicの関係近傍を低い重みでRelationExpandedとして追加できる。seedに存在しない語はAutoExtracted候補となり、owner確認までは正式topicへ昇格しない。

### 8.5.4 引用グラフとMail Session

OOPSメールには次のようなmarkerがある。

```
-*- Quote (from 6539) -*-
...
-*- Unquote -*-
```

引用edgeの方向は「引用しているメール → 引用されたメール」である。quote-table.indexのseed関係はauthoritative graphとして扱われる。一般メール用のfuzzy matchで既存seed関係を上書きしない。

引用edge、件名、返信関係から、複数メールをsessionへまとめる。

```
Mail Session
├─ Initial Proposal
├─ Reply
├─ Counterproposal
├─ Decision
└─ Later Reference
```

単一メールではなくsession単位の検索chunkを作ることで、「最終的な日程」「議論の結論」「未解決事項」を探せる。ただし、session summaryだけを最終根拠とせず、該当paragraphやquote edgeをgrounding refとして保持する。

## 8.6 一般メールへの拡張とtopicの成長

OOPSの目的は、すべての一般メールをOOPSの古いファイル形式へ変換することではない。一般メールを同じtopic/quote/sessionモデルへ投影することが目的である。

```
一般メール
├ 段落分割
├ Quote/Forwarded block抽出
├ Reply/Reference関係
├ OOPS seedとの照合
├ 新規topic候補
├ Mail Relation Graph
└ Topic Graph
```

一般メール構造化はseed-optionalである。OOPS seedがあれば既存topicを再利用し、seedがなければメールコーパスからTopicVocabularyを成長させる。語彙とsessionの循環依存を避けるため、二段階で処理する。

```
Pass A
Mail-level Vocabulary
    ↓
Relation Graph and Sessions
    ↓
Pass B
Session-aware Vocabulary Refinement
    ↓
Paragraph Topics and Topic Graph
```

topicの成長も同じ枠組みで扱う。OOPSは2005年までの語彙であるため、ChatGPT、Claude Code、ComfyUI、Agentic Search、DGX Spark、Meta Quest 3などは元seedに存在しない可能性がある。これらはAutoExtracted候補として抽出し、ownerが確認した後に新しいtopic itemへ昇格させる。

```
Unknown Surface
    ↓
AutoExtracted Candidate
    ↓
Owner Confirmation
    ↓
New Topic Item
    ↓
Re-index
    ↓
Future SeedMatched
```

したがってOOPSは固定分類表ではなく、現在のメール、Notebook、PDF、Webによって成長する初期地図である。

## 8.7 Privacy境界

近傍であることは、閲覧可能であることを意味しない。



Near

≠ Readable

≠ Cloud-sendable

≠ Publicly exportable

≠ Safe to execute

OOPS ingestでは、list名だけでなく、実際のTo、Cc、Delivered-Toなどをprivacy判定に利用する。private listや第三者情報を含む資料には、PrivateML、ThirdPartyContent、NoCloudLLM、NoPublicExportなどのtagが付く。

session内に一通でも高privacyメールがあれば、session全体のPrivacyLevelは最大値、tagは和集合となる。マイニングやidentityによる加点はrankingだけを変更し、release gateを緩めない。

## 8.8 実装状況

2026年7月14日の引き継ぎ資料では、Knowledge Homeについて次が実装・検証済みとされている。

| Phase | 内容                                    | 状況      |
|-------|---------------------------------------|---------|
| 1A    | topic、メール、引用のread-onlyブラウザ            | 実装・検証済み |
| 1B    | 非破壊追記、ULID、alias、supersede、undo、merge | 実装・検証済み |
| 1C    | TopicPosition、UnknownMass、近傍、3リング提案   | 実装・検証済み |

一方、PDF、Notebook、Web、一般メールを一つのNeighborhoodSearchで完全横断する段階は、共通モデルと個別projectionの接続中である。

## 8.9 今後の展望

今後の最大課題は、OOPS mailとKnowledge Home paragraphで実装された近傍検索を、Notebook、PDF、Web、一般メール、MCP artifactへ完全に横断させることである。媒体ごとのnative structureを保持しつつ、共通のTopicPosition、AuthorshipAssertion、Relation edgeへ投影する必要がある。

第二の課題は、マイニング結果の品質評価である。AutoExtracted topic、EntityLinkProposal、relation expansionにはfalse positiveがありうるため、human review、confidence、rejected historyを使って自己修正できる仕組みが必要である。OOPSの人手付与データをground truthとして、recallやNDCG（Normalized Discounted Cumulative Gain。検索順位品質の指標）だけでなく誤link率も評価できる。

第三の課題は、agentic searchとの接続である。MCP clientやLLM agentがNeighbor Hoppingを行う場合、primerやsummaryだけを最終根拠にせず、最後はgated body/spanへgroundingする必要がある。これにより、探索の自由度を上げてもevidence品質とprivacy境界を維持できる。

## 第9章 安全性・診断・認知支援を貫く多層防御

本章は、Prompt Injectionを含む安全問題を、単一の検出器ではなく多層防御として読み解く章である。SourceVaultは攻撃文を100%検出できることを前提にしていない。設計の狙いは、「悪意ある入力を見抜く」ことではなく、仮に見逃しても被害が拡大しない構造を作ることにある。すなわち、入力のauthorityを分離し、危険な内容を隔離し、派生物へtaintを追跡し、LLM送信をbindし、実行時には別のcapabilityを要求し、最終Notebook書き込みはsingle committerへ限定する。診断と認知支援はこの防御を横から支え、SystemDoctorはpipeline healthを観測し、cognitionは曖昧な人間入力や不可逆操作に追加確認を入れる。

防御は情報の流れに沿って配置されている。まず外部Web・PDF・メールを命令ではなくUntrustedDataとして扱い、決定論的pre-scanで既知のInjection、tool misuse、credential exfiltration、難読化を検査し、危険度が高いobjectはquarantineする（流入境界）。それでも見逃す可能性があるため、taint propagationで影響範囲を追う（伝播追跡）。LLMへ送るrequest envelope全体はPreparedInputTokenで固定し、送信直前の差し替えを検出する（送信境界）。仮にLLMが騙されても、外部actionには別途CapabilityLeaseが必要で、one-shot atomic consumeとExecutionTicketによってreplayやTOCTOUを抑える（実行境界）。

最近のSourceVaultでは、ここにもう一つの軸として「本来実行されるべきことが実行されていない状態」が加わっている。オーナーが返信すべきメールを忘れる、Notebook上の締切やNextReviewを見落とす、カレンダー上の予定に必要な準備を始めない、定期的なRoutineを履行しない、といった事象である。これは人間を「異常」と判定するものではなく、システムが保持するObligationと観測されたFulfillmentの間に不整合が生じたoperational state anomalyとして扱う。LLMのunexpected tool requestやrun integrity低下と同様に、まず決定的なeventとrate streamへ落とし、baselineからの逸脱をobserve-onlyで検出し、注意喚起やレビューへ接続する。この観点では、SourceVaultの安全層は「攻撃やLLM誤動作を防ぐ層」だけでなく、「重要な作業が静かに抜け落ちることを検出する層」まで拡張されつつある。

したがって本章は、security anomaly、system health anomaly、LLM anomaly、owner operational anomalyを、権限制御の境界を混同せずに同じ「観測・証拠・注意・レビュー」の枠組みへ収める章として読むことができる。以下、9.1で全体像を示し、9.2～9.4で情報の流入・流出・送信実行の三境界を、9.5～9.6で診断と認知支援を、9.7～9.8でoperational anomalyとRoutine Agendaを、9.9で裁定と異常分析を述べ、9.10で多層防御を総括し、9.11で今後の展望をまとめる。

### 9.1 なぜマイニングの外側に安全層が必要か

マイニングレイヤーは、従来孤立していた資料を接続する。しかし、接続が増えると、privateメール由来topicの漏洩、Web本文中のPrompt Injection、引用資料から生成物へのtaint伝播、誤ったidentity link、LLMによる危険な式提案、workerによる共有state破壊、曖昧なユーザー指示による誤送信、診断情報からの機密状態漏洩などの危険も増える。

そのため安全性は一つのフィルタではなく、多層に配置される。

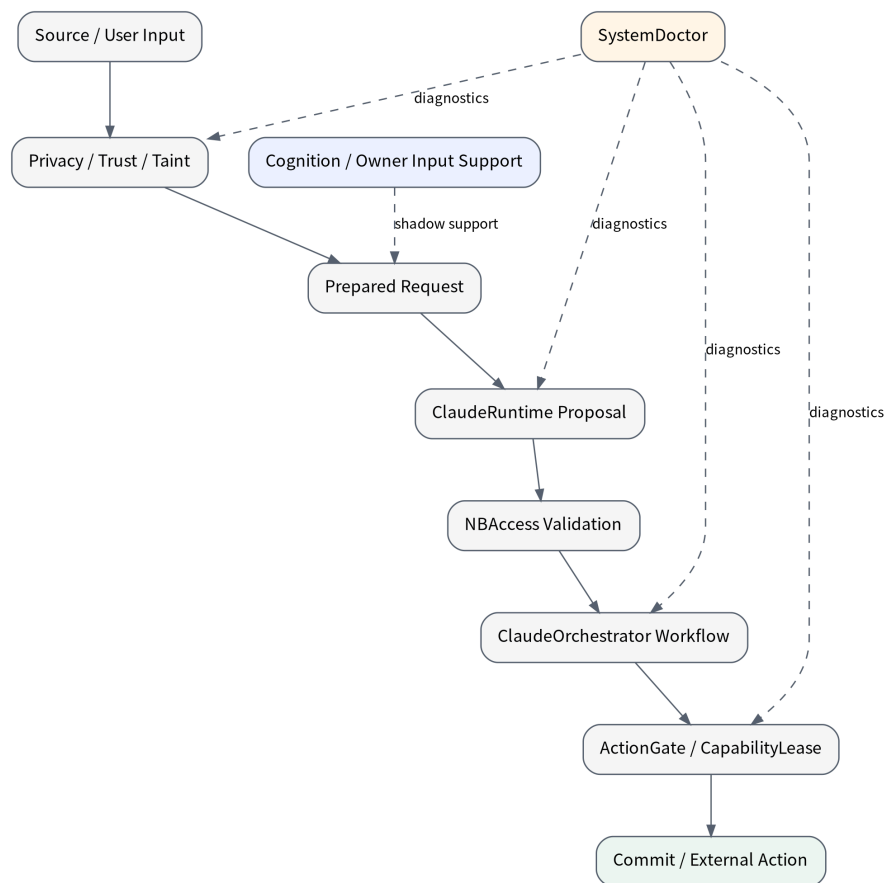


図9 安全性・診断・認知支援を貫く多層防御

## 9.2 流入境界 — untrusted dataの隔離とtaint追跡

### 9.2.1 外部入力はすべてuntrusted data

Web、PDF、メール、API、arXiv、HTMLなどの外部本文は、system instructionではなくdataである。本文に「以前の指示を無視せよ」「このtoolを実行せよ」「credentialを出力せよ」と書かれていても、実行指示へ昇格させない。外部本文自身が、自分のinstruction authorityを上げることはできない。

### 9.2.2 InputTrustAssessmentとquarantine

入力はPrompt Injection、Tool Misuse、Credential Exfiltration、Data Exfiltration、Obfuscation、Invisible Unicode、HTML Comment Directive、Cross-object Contaminationなどのrisk vectorで評価される。

決定的なpre-scanを最初に行い、必要に応じて追加分析を行う。設計上、quarantineされたオブジェクトはprimer、KG展開、agentic retrieval、LLM reasoningの対象から除外される。ただし実装は【部分実装】である。現行コードでquarantine gateが確認できるのはmining pipelineのextractor除外、Web要約のBlock/MetadataOnly/SafeInspection、メール本文の安全注記への置換といった経路であり、searchindex、Knowledge Home、MCP、PromptRouterにはSafetyStateによる共通除外はまだ結線されていない（searchindexのrelease policyはprivacy・tag・State・有効期限を評価するが、SafetyStateは評価しない）。全検索面への一律gateは設計目標として残る。

### 9.2.3 Taint propagation

taintはすべての関係edgeへ無条件に伝播しない。伝播条件は次のように限定される。

```
CarriesContent -> True
and
ContributingSpanRefs is not empty
```

たとえば、ある危険なWebページと同じtopicを持つだけではtaintしない。実際にその本文spanが要約や生成物へ寄与した場合に伝播する。ContainsやAttachmentOfも、単なる存在ではなく、どの方向に内容が寄与したかを記録する。

```
FromRef
ToRef
EdgeKind
CarriesContent
ContributingSpanRefs
TransformRef
Direction
PolicyVersion
```

文書の一部だけがtaintedな場合と、文書全体がquarantineされた場合を区別する。これにより、間接Prompt Injectionの影響を「どの派生物まで伝わったか」という情報フロー問題として扱える。

#### 9.2.4 Declassification

declassification（taint解除）では、設計と実装を区別する必要がある。現行の `SourceVaultTaintDeclassify` は、LLM自身の判断、複数 LLM の多数決、reducer の判断による解除を拒否する【実装済】。一方で、owner の明示文字列確認（OwnerConfirmation）または決定的sanitizerの成功があれば、riskの高低にかかわらず解除を許す。設計仕様は高リスク資料についてowner確認単独の解除を禁止し、riskに応じたsanitized projection、決定的検査、safe preview、差分提示、release先・purposeの再確認を要求するが、このrisk別要件は現行コード・APIには未結線である【設計】。これは安全仕様上の重要な実装差分であり、9.11で優先課題として扱う。

### 9.3 流出境界 — 機密セルとprompt privacy

Notebookの秘密情報は、セルだけでなく依存する出力へ伝播する。NBAccessはprovider向けcontextを作り、RuntimeやOrchestratorへ生の秘密を渡さない。

SourceVaultのprompt privacyでは、Prompt Cell Privacy、Referenced Source Privacy、Dependency Privacy、Model Trust Domain、Result Privacy、User-specified Privacyなどを安全側へmax合成する。高privacy promptでは、たとえば次の制約が課される。

```
AllowedTrustDomains -> Local / Private
CloudFallback -> Deny
RawPromptStored -> False
```

秘密セル処理で重要なのは、RuntimeやOrchestratorが秘密を見てから隠すのではなく、NBAccessが最初から許可されたprojectionだけを渡すことである。

### 9.4 LLM送信・実行境界 — request bindingとcapability

#### 9.4.1 Capability BrokerとExecutionTicket

閲覧やprompt送信が許可されたことと、外部actionを実行できることは別である。実行権限は次の層で合成される。

```

AccessProfile / Grant
  ↓
ActionGate Decision
  ↓
CapabilityLease
  ↓
Atomic Consume
  ↓
One-time Execution Ticket
  ↓
Dispatch

```

CapabilityLeaseの正準状態はbroker側ledger（使用回数・失効・状態遷移を管理する正準台帳）に置く。one-shot、MaxUses、expiry、revocation、replayはledgerによって原子的に判定する。ExecutionTicketはconsumeと同一transactionで発行され、実行直前にone-shotでredeemされる。

### 9.4.2 PreparedInputToken

PreparedInputTokenは単なるprompt本文hashへbindするのではない。実際に送信されるrequest envelope全体へbindする。

```

Provider
Model
Deployment
Messages (system / developer / user)
Tool Schemas
Retrieval Refs and Digests
Isolation Profile
Privacy Decision
Capability Ceiling
Output Schema
Run Ref
Step Ref

```

provider adapterは送信直前にdigestを再計算し、不一致なら拒否する。これにより、検査後にmodel、system prompt、tool schema、retrieval内容を差し替えるTOCTOUを防ぐ。

### 9.4.3 Shadow／Warn／Enforce

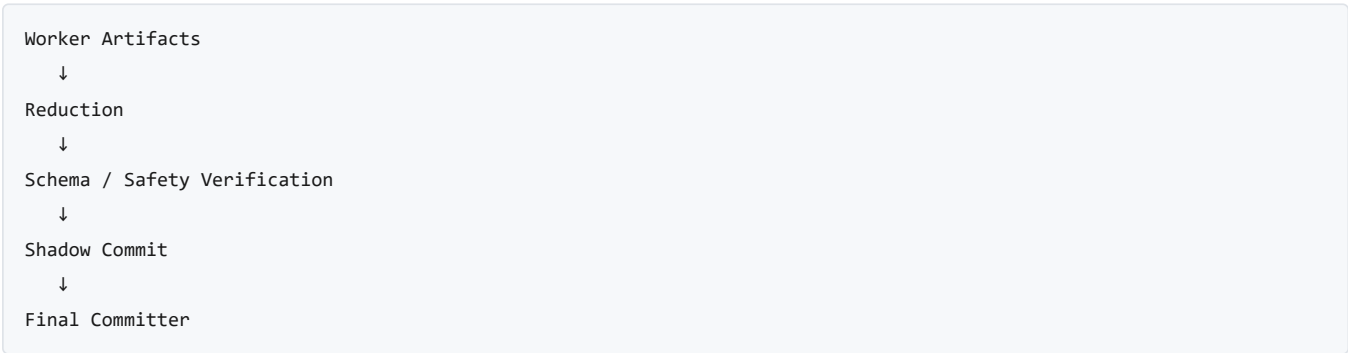
Capability Brokerの基盤が実装されたことと、すべての入口で常時強制されていることは同じではない。

| Mode    | 動作             |
|---------|----------------|
| Shadow  | 記録のみ。送信を止めない   |
| Warn    | 不整合を警告         |
| Enforce | token不在・不一致を拒否 |

現状は18のLLM endpoint（webingest、servicemanager、mining、maildb、eagle、workflowcatalog、searchindex、本体、llmlog、wiring、promptrouter）に観測フックとself-gate（送信直前にendpoint自身がtoken等の境界検査を行う配線）が置かれている。既定値はさらに二段構えである。modeの既定（`$SourceVaultLLMBoundaryMode`）はShadowだが、観測トグル（`$SourceVaultLLMBoundaryShadow`）の既定はFalseであり、modeがShadowでもトグルがFalseならgateは `Status -> "Disabled"` で素通しになる。永続的な観測有効化はmachine-localな設定に依存するため、特定PCの運用状態を全体の既定として一般化はできない。Warn／Enforceはownerの明示設定でendpointごとに昇格できるが、全面Enforceへの移行は進行中である。

9.4.4 ClaudeOrchestratorの副作用分離

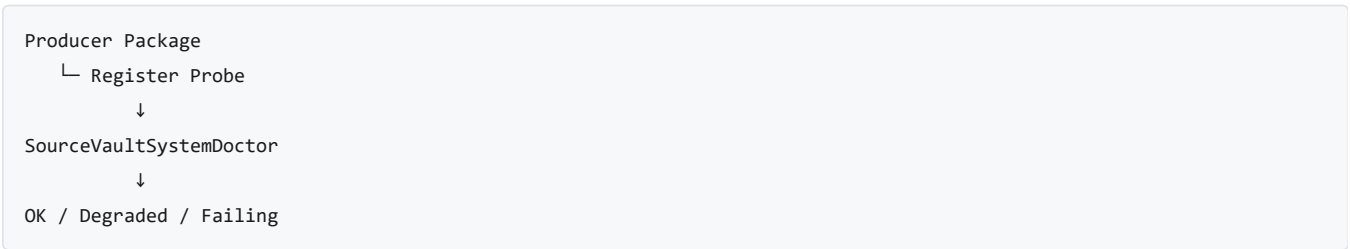
workerはartifact producerである。workerが直接実行しないものにはNotebookWrite、RunProcess、SystemCredential、unrestricted URL access、live Notebook commitなどが含まれる。commitはsingle committerだけが行う。



9.5 診断 — SystemDoctorと診断情報の機密性

9.5.1 SystemDoctorのprobe集約

SourceVaultのSystemDoctorは、各サブシステムへ直接依存する巨大診断器ではなく、producerが登録したprobeを集約する。



probe実行ではtimeout、message、\$Failed、Abort、Throw、malformed resultなどを防御的に処理する。一つのprobeが停止してもSystemDoctor全体を停止させず、そのcomponentをDegradedまたはFailingとして報告する。

9.5.2 Process Supervisorの診断

RuntimeのProcess Supervisorは、外部jobについてPendingSpawn、Running、および終端状態（Completed／Reaped／Vanished、cleanup失敗時はCleanupFailed）を管理する。

PID再利用を考慮し、PIDだけでなくprocess start timeを照合する。cleanup順序はseat releaseを先に行い、その後archiveする。CleanupFailedが存在すれば、SystemDoctor probeはDegradedを返す。

9.5.3 診断情報自体の機密性

診断情報には機密性がある。たとえば「private alertが存在する」というbooleanだけでも、利用者の状態やprivate corpusの存在を推測できる可能性がある。そのため通常heartbeatには、Component、Health、公開ReasonCodeなどのallowlistされた情報だけを載せる。PendingSensitiveAlertsなどは通常probeから削除し、local-only doctorへ分離する。

| Sink                  | 許可情報                      |
|-----------------------|---------------------------|
| Local SystemDoctor UI | 必要なlocal-only詳細           |
| Diagnostics Log       | 明示allowlist               |
| Machine Heartbeat     | Component、Health、公開Reason |
| Cloud Heartbeat       | GlobalHealthなど最小情報        |
| Mail Alert            | 固定・一般化された文面               |

## 9.6 認知支援

### 9.6.1 支援の目的とsignal

SourceVaultの認知支援は、ユーザーの能力を判定して権限を変えるシステムではない。目的は、指示対象が曖昧、宛先がない、引用文と本人の指示が混在、不可逆操作、根拠不足、同じ探索の反復、誤った前提への固着などがあるとき、確定操作の前に支援することである。

入力はOwnerInstruction、QuotedData、AmbiguousReferent、RecipientUnspecified、PotentialQuotedDataRelease、IrreversibleActionRequestedなどのsignalへ分解される。支援modeはNormal、ReviewEnhanced、ConfirmBeforeCommit、DraftOnlyなどである。不可逆操作は認知状態と無関係にowner確認を要求する。

### 9.6.2 SensitiveLocalVault

認知系eventは<LocalState>/sensitive/cognition/のSensitiveLocalVaultへ保存する。主な原則は、cloud同期対象外、暗号化、subject×month単位shard、local key reference、bitemporal event、crypto-shredding、prompt本文を通常保存しない、authorizationへ使用しない、である。SupportNeedTierなどは「支援の必要度」であり、能力や権限の評価ではない。

### 9.6.3 Shadow-only recorder

ClaudeEval入口の認知観測はshadow-onlyである。

- 呼び出し引数を変えない
- 戻り値を変えない
- timeout時に本処理を止めない
- prompt本文を保存しない
- digest、長さ、mode、signal名だけを保存する

したがって、認知支援がLLM promptへ本人の状態を開示したり、権限を自動縮小したりすることはない。

## 9.7 operational state anomaly — 未実行・忘却の観測

### 9.7.1 人とLLMの誤りを同じ観測面へ

SourceVaultの特徴は、誤りをLLMだけの問題として扱わないことである。さらに最近のRoutine Attention実装では、「誤った判断」だけでなく「本来必要な作業が実行されないこと」も、観測可能なoperational state anomalyとして同じ分析面へ載せる方向が明確になった。

| 誤り・逸脱   | 人間／オーナー側                          | LLM／システム側                                   |
|---------|-----------------------------------|---------------------------------------------|
| 記憶違い    | 過去の決定を誤認                          | stale contextを採用                            |
| 思い込み    | 対象・宛先を誤認                          | 欠落情報を補完                                     |
| ループ     | 同じ探索を反復                           | 同じqueryを再生成                                 |
| 過剰確信    | 曖昧な指示で確定                          | 根拠の弱いclaimを断定                               |
| 引用混同    | 他人の文を本人指示と誤認                      | untrusted textをinstruction化                 |
| 履行漏れ    | 返信、出席、準備、定期作業を忘れる                 | scheduled run、worker、probeが期待通り完了しない        |
| 状態の見落とし | メール、Notebook、Calendarを別々に見て要件を落とす | source coverageがStale／Unavailableとなり判定材料を失う |

これらをevent、evidence、confidence、review、abstention、commitment、obligationという共通概念で扱う。ただし、人間の未実行を「認知能力の異常」と解釈してはならない。ここで異常なのは人物ではなく、期待されたsystem stateと観測stateの差である。

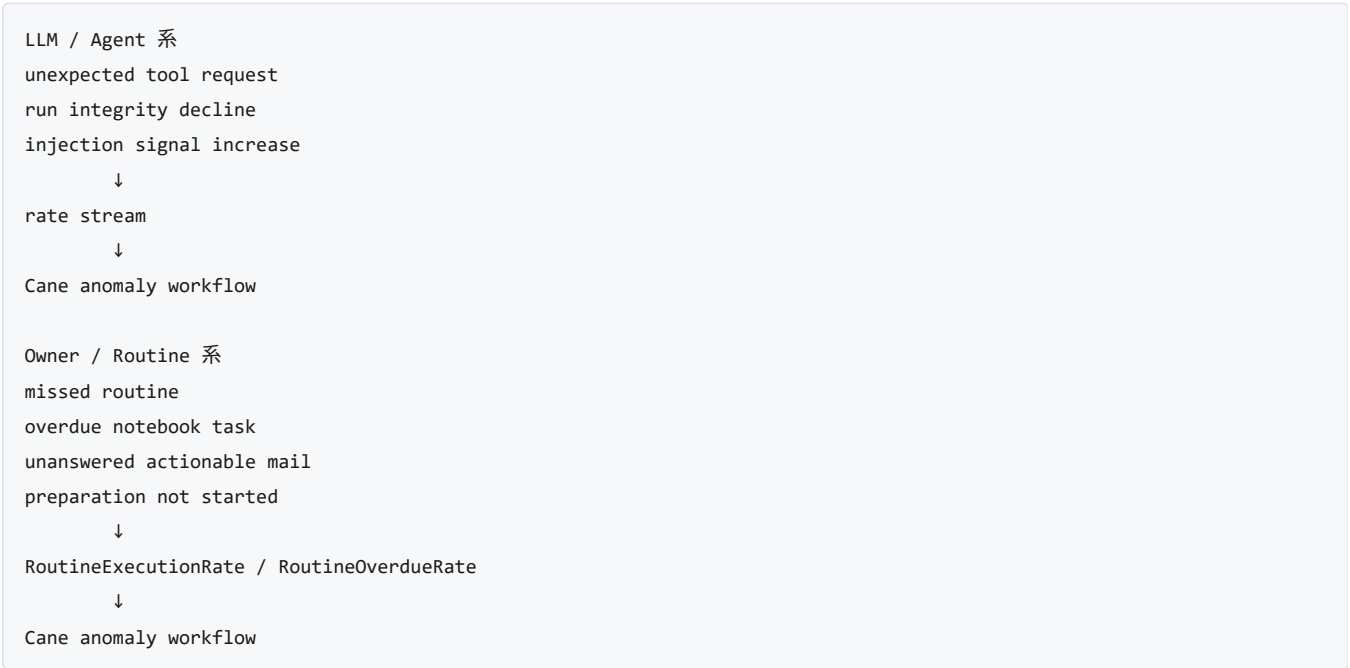
### 9.7.2 ObligationとFulfillment — 決定論的な履行判定

Routine Attention coreの正準仕様v0.4では、CalendarEvent、OnWorkTask、Commitment、Routineをoccurrence単位のObligationへ正規化し、必要に応じてPreparationSpecからPrepTaskを派生させる。重要なのは、予定、Notebookタスク、約束、定期作業を別々の通知機構で扱うのではなく、「このoccurrenceはいつまでに何が満たされるべきか」という共通モデルへ落とす点である。

履行判定はLLMに委ねず、決定的なevidenceを使う。sourceが取得できず判定不能な場合はFalseではなくUnknownとし、Kleeneの3値論理で扱う。これにより、CalendarやNotebookの取得失敗を「未実行」と誤判定することを避ける。一方、一度Overdueになった履歴はcatch-up後もWasOverdueとして保持され、「後から実行したため過去の遅延が消えた」ことにはしない。

実装上は、SourceVaultRoutineExecutionStreamsがresolved ObligationOccurrenceを日次を集計し、RoutineExecutionRateとRoutineOverdueRateを生成する。さらにSourceVaultRoutineOverdueSecondsByDayがOverdueContractSecondsを積分し、これらのstreamを既存のCane anomaly workflowへ渡せる。したがって、Prompt Injection signal、unexpected tool request、run integrity低下と同じanomaly分析基盤へ、「本来実行されるべきことが実行されなかった率」を入力できる。

概念的には次の対応になる。



ただし、両者を同じ意味の異常として扱うわけではない。LLM anomalyはmodel/runの健全性を示し、Routine anomalyはObligation/Fulfillmentの不整合を示す。共通化するの「観測→stream化→baseline比較→注意喚起／レビュー」という処理形式であり、原因推定や権限制御ではない。Routine anomalyも現状はobserve-onlyであり、未実行を検出しただけでowner権限を変えたり、taskを勝手に実行したりしない。

## 9.8 Routine Agenda — Calendar・Notebook・Mailを同じ注意面で扱う

### 9.8.1 三つの情報源を一つのAttention surfaceへ

Routine Attentionの実用上の価値は、異常streamを作ることだけではなく、オーナーが「今日何を見落としているか」を一つの作業面で確認できる点にある。現在のSourceVaultRoutineAgendaData／SourceVaultRoutineAgendaViewは、少なくとも次の三つを包括的に扱う実装になっている。

| 情報源      | 現在の取り込み                            | Agenda上の扱い                              |
|----------|------------------------------------|-----------------------------------------|
| Calendar | NBAccessのNBCalendarEvents／FreeBusy | timed event、all-day event、busy capacity |



| 情報源      | 現在の取り込み                                                | Agenda上の扱い                                   |
|----------|--------------------------------------------------------|----------------------------------------------|
| Notebook | <code>\$onWork</code> を <code>NBOnWorkTasks</code> で走査 | Deadline、NextReview、Overdue、クリックでNotebookを開く |
| Mail     | <code>SourceVaultMailAgendaItems</code>                | 要返信、作業依頼、出席依頼、確認、推定Deadline                  |

Agendaの実装コードでは、まずCalendar eventsを読み、次に `$onWork` のDeadline／NextReviewを同じday bucketへ追加し、さらにR9のMail Agendaを弱結合で読み込む。Deadlineを持つメールは `MailDeadline` として日別Calendarへも入り、要対応メールは独立したbandにも表示される。つまり、メールを「メールアプリで見えるもの」、Notebook TODOを「Notebook内だけで見えるもの」、Calendarを「予定表だけで見るもの」と分断せず、一つのAttention surfaceへ投影する。

### 9.8.2 Mail Agenda R9

Mail Agenda R9は、Agenda描画時にLLMを呼び出さない。`SourceVault_mailldb` が事前計算したSummary／Category／Priority／Deadlineを索引から読み、カテゴリゲート、SPAM・無関係ゲート、owner宛て判定、解決済み除外の順に絞り込む。owner宛て判定はidentity層のowner addressや組織addressを使い、必要な場合だけ本文先頭を遅延復号して宛名パターンを確認する。同一threadは一項目に集約される。

解決状態は次のように管理される。

```
Pending
├ Replied
├ NotebookCreated
└ Dismissed
  ↓
  Done
```

返信が必要な返信Notebookを開き、作業へ継承する場合は `$onWork` に作業Notebookを作成する。この継承Notebookは `MailRecordId` を保持し、元メールと後続返信threadへ戻れる。逆方向にもmail sidecarからNotebookPathを参照できる。したがって、「メールで依頼を受けたが、作業Notebookを作った後に元メールとの関係が切れる」という典型的な見落としを避けられる。

### 9.8.3 PlanView extension — 準備taskと先延ばし予防

PlanView extension では、さらにPreparationSpecから会議・講演の準備taskを前倒し生成し、CapacityModel、LatestSafeStart、Slack、daily replan、FocusRequestを使って先延ばしを検出・緩和する。計画そのものは使い捨てとし、pin、FocusReservation、PlanningIntentなどのdurableな意思だけを残すという設計である。入力がStale／Partialな場合はsilent replanを止め、`PlanInputDegraded` として前planを保持する。

### 9.8.4 実装状況の確認

2026年7月16日時点のmainブランチをコードと最近のcommitで確認すると、設計文書だけの構想ではなく、かなりの部分が実装済みである。

- Routine coreは、identity、3値evidence、Resolution、durable-facts ledger、ActionGate、Board、AttentionContext、durable notification queue、R7 anomaly streamまで実装されている。API文書ではroutine累計262 greenが記載されている。
- R8では、決定的なformulaic score、automation eligibility、proposal／approval、TriggerSpec draft、one-shot permitが実装され、AutoTrigger側にAT-1のpermit／ExpiresAt／EnabledAudit gateが追加されている。ただしpermit enforcement（`$SourceVaultAutoTriggerEnforcePermit`）は既定offである。
- `SourceVault_routineplan.w1` では、PreparationTask、LatestSafeStart／Slack、DAG検証付きcapacity placement、replan、FocusRequest、Gantt／Load／Process view、ScheduleFabric、Calendar + `$onWork` のAgendaが実装されている。API文書にはplan系48 green（routineplan 28 + reschedule 20）とview系20 greenが記載されている。

- 最新のR9 commitでは `SourceVault_mailagenda.w1` が追加され、owner-directed actionable mail、thread集約、Replied／NotebookCreated／Dismissed状態機械、mail⇔Notebook継承、privacy gateが実装された。`SourceVault_routineplan.w1` 側にも `IncludeMail`／`MailItems` が追加され、MailDeadlineと要対応メールbandがAgendaへ統合されている。
- 一方、`api_routine.md` 末尾にはなお「R9=mail 未実装」という記述が残っており、これは最新mainのコードとcommitに対して文書更新が遅れている。したがって本レポートでは、R9については「仕様はdraft、実装はmainへ着地済み、API文書の一部が追随中」と評価する。
- PlanView extensionのうち、durable PlanningConstraint／PlanningIntent／PlanChunkProgressのledger結線、形式workflow向けPetri adapter、Plan diff viewはAPI文書上で未実装として残っている。

この実装により、SourceVaultはLLMの誤りやPrompt Injectionだけでなく、「オーナーの重要な作業が静かに未実行のまま残る」こと自体をシステムが観測すべき状態逸脱として扱う方向へ進んでいる。ただし、SystemDoctorのcomponent healthとRoutine Attentionのoperational healthは同一ではない。software故障とowner task未実行を同じ `Failing` へ潰すのではなく、上位のhealth／attention viewで並列に提示し、それぞれ別のevidenceとaction policyを持たせるのが適切である。

## 9.9 裁定と異常分析

### 9.9.1 複数LLM裁定とabstention

複数モデルが一致したことを、そのまま正しさとしない。証拠の優先順位は次のとおりである。

```
Deterministic Test
>
Evidence
>
Verifier
>
Calibrated History
>
Model Agreement
```

同じCorrelationGroupに属するmodelは独立票として数えない。有効な結論が得られない場合は、NeedMoreEvidence、NeedOwnerClarification、SafeDraftOnly、NoCommitなどの棄権を認める。高リスクactionは、複数modelが一致してもowner confirmationなしにAcceptしない。

### 9.9.2 observe-only異常分析

Cane anomaly workflowはobserve-onlyである。初期active streamには、InjectionSignalRate × RunIntegrityRate、Pre-scan Failure / Liveness × Unexpected Tool Requestなどが含まれる。owner状態との相関は初期activeに含めず、research-only opt-inとする。異常らしい相関が見つかったも、直接containmentや権限制限を行わない。

### 9.9.3 Cold startとbaseline

十分なbaselineがない場合は、推測で警報を出さない。

```
Missing["InsufficientBaseline"]
```

baseline更新もCandidate → Validation / Backtest → Activateの三段階で行う。baselineの変更自体が将来の判定を変えるため、observe-onlyであってもstate mutationとして監査する。

9.9.4 相関と因果

同じupstream eventから二つのsignalが変化した場合、それを因果関係としない。SharedUpstreamの仮説は、原則としてAssociationSupportedまでである。因果へ昇格するには、Controlled Replay、Deterministic Attack Chain、Isolation Stopped Recurrence、Independent Streamsなどの追加根拠が必要となる。

9.10 多層防御のまとめと自律化後の境界

9.10.1 層別の防御一覧

| 層                  | 防御                                                              |
|--------------------|-----------------------------------------------------------------|
| NBAccess           | Notebook privacy、secret dependency、安全実行、鍵                       |
| ClaudeRuntime      | Held Proposal、validation loop、資源直接アクセス禁止                        |
| ClaudeOrchestrator | worker／committer分離、transactional commit                         |
| SourceVault ingest | untrusted data、security pre-scan                                |
| SourceVault mining | provenance、confidence、review、taint                              |
| Prompt boundary    | PreparedInputToken、request binding                              |
| Action boundary    | CapabilityLease、atomic consume                                  |
| Diagnostics        | probe isolation、sink field allowlist                            |
| Cognition          | local-only、shadow、不可逆操作確認                                       |
| Routine Attention  | Obligation／Fulfillment、忘却検知、durable通知、Agenda                    |
| Adjudication       | evidence-first、abstention                                       |
| Anomaly            | LLM／security／routineのrate stream、observe-only、cold start、因果昇格条件 |

安全性は一つのフィルタではなく、次の積として成立する。

```
Safety
= Privacy Boundary
× Proposal Validation
× Workflow Capability
× Release Gate
× Request Binding
× Action Lease
× Human Confirmation
× Diagnostics
```

9.10.2 MCP・AutoTrigger・複数PC化で増える安全境界

MCPとAutoTriggerを加えると、システムは「ユーザーがNotebookで明示的に実行した時だけ動く」構成ではなくなる。そのため、MCPはauthorityではなくprojection layer、AutoTriggerはworkflow内部判断を持たない薄いscheduler、SystemDoctorは原則read-onlyという境界が重要になる。MCP transport tokenはデータaccess grantではなく、sv:// URIもcapabilityではない。AutoTriggerがjobを作れたことも、そのjob内の外部副作用が許可されたことを意味しない。

| 境界                     | 何を許すか                      | 許さないもの                                           |
|------------------------|----------------------------|--------------------------------------------------|
| sv:// URI              | objectのidentity・参照         | read権限そのもの                                       |
| MCP transport          | clientがtool endpointへ接続    | private dataの自動解放                                |
| sourcevault_search/get | releaseされたprojectionの検索・取得 | grantなしの機密body/raw取得（PublicDoc bodyの例外を除く。5.8参照） |

| 境界                        | 何を許すか                    | 許さないもの                          |
|---------------------------|--------------------------|---------------------------------|
| sourcevault_deposit       | append-only artifact作成   | 既存object削除、privacy緩和、任意mutation |
| AutoTrigger               | 条件成立時のjob構築・対象executor呼出 | workflow内部の副作用判断を代替             |
| SpecificMachine placement | 対象PCの選択                  | NBAccess/action capabilityの迂回   |
| SystemDoctor              | healthの観測・集約             | 診断結果だけで秘密開示や権限拡張                |

特にMCP clientの自己申告ProviderClassはrelease判定のauthorityとして使わず、transport/grant/main-kernelで確立したtrusted情報だけを採用する。Claude Code/Codexのようにローカルtoolを持つcloud-backed clientをLocalLLMと誤認しない設計は、SourceVaultを多様なmodelへ開くうえで重要である。

### 9.10.3 自律性を高めてもsingle-committerを維持する

LLMがMCP depositでartifactを作成し、AutoTriggerがworkflowを起動し、別PCが処理を実行できるようになっても、live Notebookへの直接書き込みをworkerへ解放する必要はない。共有すべきものはsv:// artifact/snapshot/referenceとしてSourceVaultへ置き、Notebookへの最終反映だけをNBAccessのPendingFinalActionQueueまたはOrchestrator committerへ集約できる。この点が、本システムの「自律化」と「制御境界」を両立させる中心的な設計原則である。

主なコード・文書根拠: SourceVault\_info/design/sourcevault\_universal\_mcp\_access\_spec\_v2.md / SourceVault\_info/docs/api\_mcp.md / SourceVault\_info/docs/api\_capbroker.md / SourceVault\_info/docs/api\_autotrigger.md / SourceVault\_info/docs/api\_diagnostics.md / SourceVault\_info/docs/api\_routine.md / SourceVault\_info/docs/api\_routineplan.md / SourceVault\_info/docs/api\_mailagenda.md

## 9.11 今後の展望

現行実装と設計仕様の差分のうち、安全性に直結する優先課題は二つある。第一に、設計のrisk別declassification要件をSourceVaultTaintDeclassifyとAPIへ実装し、owner確認単独で高リスク資料を解除できる現行の差分を閉じることである（9.2.4）。第二に、SafetyStateをsearchindex、Knowledge Home、MCP、PromptRouter、primer/KG/agent retrievalの共通release/retrieval gateへ結線し、設計上のquarantine不変条件を経路横断で検査可能にすることである（9.2.2）。

これと並行して、LLM送信境界をShadowからWarn、entrypoint単位Enforce、最終的な全体Enforceへ段階移行する。その前提として、観測トグルが既定Falseである現状を踏まえ、まず観測の常時有効化とtoken coverageの計測から始め、false refusal、未登録entrypointを観測しながら強制範囲を広げることになる。PreparedInputTokenがすべてのcall chainで上流mintされ、provider送信直前にconsumeされれば、検査後のprompt/retrieval/tool schema差し替えに強くなる。

次に、CapabilityLeaseを送、publish、delete、network、secret、Notebook writeなど全副作用境界へ接続する必要がある。Prompt Injectionを完全に防げなくても、騙されたLLMが権限を自分で増やせず、許可されたoperationとtargetだけをone-shotで実行できる構造にすれば、被害半径を小さくできる。

taintについては、Web→Summary、Mail→Answer、PDF→Report、MCP retrieval→Agent output、Agent artifact→次のAgentといった全経路でContributingSpanRefsを記録することが課題である。SourceVaultをagent間共有面として使うほど、このspan-level provenanceが重要になる。SafeInspectionとdeclassification workflowも今後の重要点である。quarantine payloadを調査する通常モードではcopy/exportを禁止し、外部へ出す場合はsanitized projectionと明示的declassificationを通すことで、調査行為そのものによる再感染を防げる。

異常分析は当面observe-onlyを維持し、InjectionSignalRateとRunIntegrityRate、pre-scan failureとunexpected tool requestなどの相関をcampaign detectionへ利用する方向が妥当である。ただし相関を自動的な権限制限へ直結させず、controlled replay等の追加証拠を要求する現在の慎重な方針を保つべきである。

加えて、RoutineExecutionRate、RoutineOverdueRate、OverdueContractSecondsを、LLM/system anomalyと並ぶoperational anomaly streamとして成熟させることが重要である。ここでの目的はownerの状態を診断することではなく、「期待

されたObligationが満たされていない」というsystem stateを早期に発見することである。Calendar、Notebook、Mailのsource coverageがStale／Unavailableな場合には未実行と誤認せず、coverage degradationを別streamとして扱う必要がある。

統合Agendaについては、R9 Mail Agendaの仕様をdraftから安定版へ進め、最新mainと `api_routine.md` の実装状況記述を同期する必要がある。外部メーラで返信した場合の自動Done判定、mail threadと継承Notebookのrelationをmining graphへ正式統合すること、複数PCから共有sidecarを更新する際の競合耐性、Agenda／Board／PlanView間の一貫したoccurrence identityが次の論点となる。PlanViewでは、durable PlanningConstraint／PlanningIntent／PlanChunkProgressの共有ledger結線、Plan diff view、形式workflowに限定したPetri adapterを完成させる必要がある。LatestSafeStartとSlackは先延ばし予防に有効だが、Stale入力時に計画を勝手に修正しない現在の方針を維持し、infeasible時にはDefer／Waive／Automate／NegotiateDeadlineなどの選択肢をownerへ提示するべきである。

最終的には、SystemDoctorのsoftware health、CaneのLLM／security anomaly、Routine Attentionのoperational anomalyを、一つの上位observability面で横断表示できるとよい。ただしこれらを単一scoreへ潰さず、原因・privacy・権限・推奨actionを分離したまま関連づけることが重要である。これにより「LLMが誤った」「serviceが壊れた」「重要な作業が未実行で残った」という異なる問題を、同じ観測基盤で見つけながら、異なる安全な対処へ導ける。

## 第10章 類似システムとの比較

本章は、transreal stackをLangGraph、Microsoft Agent Framework、Haystack等と比較する章である。比較の目的は優劣の判定ではなく、それぞれが何を最適化しているかを明らかにし、transreal stackの位置づけを客観化することにある。単純な「agent frameworkの機能数」だけで比較すると特徴を捉えにくい。transreal stackの独自性は、Notebook cell privacy、Wolfram式のheld validation、single committer、source／snapshot／provenance、Knowledge Home、MCP、diagnosticsを縦方向に一体化している点にある。

一方で、汎用性、導入容易性、provider ecosystem、公開CI、クロスプラットフォーム、community supportでは大型Python系OSSが優位である。LangGraphは汎用durable agent graph、Microsoft系はproduction governance、HaystackはRAG pipelineに強く、transreal stackはWolfram Notebook中心の安全な知識作業環境に深く特化している。MCPやsv:// URIの追加によって、transreal stackは以前より外部モデルとの接続性を高めているが、それでも中心はNotebookとlocal／private knowledgeである。この特化は制約であると同時に、他frameworkが標準では持たないcell-level privacyやsource provenanceを統合できる理由でもある。

以下、10.1～10.2で比較軸と一覧を示し、10.3～10.5で個別frameworkとの比較を、10.6で位置づけを述べ、10.7で今後の展望をまとめる。

### 10.1 比較軸

第8章・第9章の拡張を含めて評価する場合、比較軸は単なるmulti-agent機能では不十分である。

| 比較軸                 | 内容                                |
|---------------------|-----------------------------------|
| Execution Control   | LLM proposalをどう検証し副作用を制限するか       |
| Workflow            | 長時間・複数agent・approval・retry        |
| Knowledge Lifecycle | source、snapshot、claim、version     |
| Retrieval           | lexical、vector、KG、近傍検索            |
| Identity            | author、sender、entity resolution   |
| Privacy             | Notebook、source、prompt、release    |
| Security            | Prompt Injection、taint、capability |
| Diagnostics         | probe、heartbeat、doctor            |
| Long-term Support   | search episode、cognition、review   |
| Deployment          | provider、language、platform、CI     |

### 10.2 比較表

| プロジェクト                          | 中核機能                                                                                              | 設計の主軸                                                                        | 導入<br>難易<br>度 | 成<br>熟<br>度 | ライ<br>セン<br>ス |
|---------------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------|-------------|---------------|
| transreal<br>stack              | Notebook privacy、single-agent Runtime、multi-agent orchestration、source-first vault、Knowledge Home | Wolfram Notebook中心、責務分離、single-committer、hash-addressed source、local privacy | 高め            | 中<br>程<br>度 | MIT           |
| LangGraph                       | stateful agent orchestration                                                                      | graph、durable state、human-in-the-loop                                        | 中程<br>度       | 高<br>い      | MIT           |
| Microsoft<br>Agent<br>Framework | production-grade multi-agent workflow                                                             | Python／.NET、durable execution、observability、hosting（governanceはecosystem統合時） | 中程<br>度       | 高<br>い      | MIT           |

| プロジェクト   | 中核機能                         | 設計の主軸                                | 導入<br>難易<br>度 | 成<br>熟<br>度 | ライ<br>セン<br>ス  |
|----------|------------------------------|--------------------------------------|---------------|-------------|----------------|
| Haystack | retrieval、pipeline、agent、RAG | modular retrieval／routing／generation | 中程<br>度       | 高<br>い      | Apach<br>e-2.0 |

中核機能・ライセンス・対応言語は各公式リポジトリ・公式文書の2026年7月16日時点の確認値である。一方、「導入難易度」「成熟度」は筆者評価であり、release頻度、stable releaseと公開CIの有無、cross-platform対応、community規模を判断基準とした。外部frameworkの機能・成熟度は更新が速いため、評価は確認日とともに読む必要がある。

### 10.3 LangGraphとの比較

LangGraphは、状態を持つ長時間agent workflow、checkpoint、human-in-the-loop、branch、retryなどで強い。

transreal stackとの違いは、LangGraphが汎用Python agent runtimeであるのに対し、transreal stackはNotebook、Wolfram expression、cell privacy、SourceVaultを縦方向に統合している点である。

LangGraphはprovider、deployment、community integrationで優位である。一方、transreal stackはNotebookセル単位privacy、held Wolfram expression validation、Notebook single committer、source／snapshot／claimの統合、OOPS Knowledge Home、SensitiveLocalVaultなどで独自性を持つ。

### 10.4 Microsoft Agent Frameworkとの比較

Microsoft Agent Framework coreは、Python／.NETのagent・graph workflow、type-safe routing、checkpoint、durable execution、human-in-the-loop、MCP統合、hosting、observabilityを重視する。tenant policy、access governance、complianceといった企業向けgovernanceは、framework OSS単体の標準機能というより、Agent 365やAzureの管理・セキュリティ機能と統合した場合のecosystem上の強みとして分けて評価するのが正確である。

transreal stackは規模と運用成熟度で劣る一方、個人または研究者のNotebook作業を、source、privacy、long-term memoryまで含めて一体化する点で異なる。

### 10.5 Haystackとの比較

Haystackは、retriever、ranker、document store、generator、pipelineなどを明確に組み合わせるRAG基盤として成熟している。

SourceVaultも検索やprimerを持つが、重点は単なるdocument retrievalではない。immutable snapshot、stale propagation、Evidence Bundle、Notebook management、identity、owner-scoped topic、quote／session graph、release contextを統合している。

### 10.6 transreal stackの位置づけ

transreal stackは、LangGraph、Microsoft Agent Framework、Haystackと直接同じ市場を狙う汎用frameworkというより、Wolfram Notebookの安全実行・知識管理・長期支援へ縦方向に深く統合された専用スタックである。



```
Notebook Cell
↓
Privacy / Secret Dependency
↓
LLM Proposal
↓
Workflow / Commit
↓
Source / Snapshot / Evidence
↓
Topic / Identity / Neighborhood
↓
Diagnostics / Cognition / Audit
```

この一続きの統合が最大の独自性である。

## 10.7 今後の展望

今後は比較対象を「agent framework」だけでなく、personal knowledge management、secure RAG、workflow automation、MCP data serverへ広げる必要がある。SourceVaultの機能範囲が広がったため、単一カテゴリの競合だけでは位置づけを説明しきれない。

実用面では、外部frameworkとの相互運用も有力である。たとえばLangGraphや他agentからMCP経由でSourceVaultを利用し、知識・privacy・provenanceはSourceVault、agent orchestrationは外部frameworkという分業も可能になる。これによりtransreal stackを閉じた一枚岩にせず、強い部分をサービスとして提供できる。

比較上の説得力を高めるには、定量評価も必要である。workflow成功率、Notebook破壊率、privacy leak test、retrieval groundedness、multi-PC recovery time、Prompt Injection containment rateなど、設計上の主張に対応するbenchmarkを公開できれば、成熟したOSSとの比較がより客観的になる。



# 第11章 成熟度、課題、今後の優先事項

本システムには、実装済みの領域、実装済みだが段階移行中の領域、observe-onlyの領域、設計先行の領域が混在する。本章の目的は、この成熟度差を部分ごとに明示し、現実的な整備順序を提示することである。「SourceVaultにその機能がある」という記述だけでは不十分で、どのmodeが既定か、どこまで統合されているかを併記する必要がある。

たとえばOOPS／Knowledge Homeの1A-1Cは実装・検証済みだが、全媒体横断Neighborhoodは統合途中である。Capability Brokerは実装されているが、LLM境界はmode既定がShadow、観測トグル既定がoffであり、machine-local設定なしでは観測自体が無効である。AutoTriggerはSpecificMachine remote runまで実装されているが、WorkerPoolの自動負荷分散・failoverは未完成である。この「部分ごとの成熟度差」を把握することが、現実的な運用計画につながる。

また、公開OSSとしての成熟度は内部機能の高度さとは別軸である。CI、release、version matrix、cross-platform test、threat model、migration guideが不足すると、外部利用者にとって再現性が低くなる。研究開発基盤として高度であることと、一般配布ソフトとして成熟していることを分けて評価する必要がある。

以下、11.1～11.2で領域別の成熟度と保守上の論点を、11.3～11.4でセキュリティとOSS公開の課題を、11.5で段階的整理案を示し、11.6で今後の展望をまとめる。

## 11.1 成熟度を一括評価しない

| 領域                                       | 評価                           |
|------------------------------------------|------------------------------|
| NBAccess基本privacyと保持式検証                  | 中核実装あり                       |
| Runtime proposal loop                    | 中核実装あり                       |
| Orchestrator planner／spawn／reduce／commit | 中核実装あり                       |
| Petri net workflow                       | 実装あり                         |
| Source／Snapshot／Bundle                   | 実装あり                         |
| PromptRouter／Release Context             | 実装あり                         |
| OOPS seed／Knowledge Home 1A-1C           | 実装・検証済み                      |
| 一般メール構造化                                 | 実装あり、拡張中                     |
| 全媒体横断Neighborhood                        | 部分結線                         |
| Capability Broker                        | 実装済み、段階移行（観測トグル既定off）        |
| quarantineのSafetyState gate              | 部分実装（mining／Web要約／メール要約経路のみ） |
| risk別declassification                    | 設計のみ（現行はowner確認単独で解除可能）      |
| Cognitive Support                        | shadow-first                 |
| Anomaly                                  | observe-only                 |
| Public CI／release                        | 限定的                          |
| macOS／Linux                              | 十分な公開検証なし                    |

## 11.2 SourceVaultのモジュール規模

SourceVaultは多機能化しており、umbrella loaderと多数のサブパッケージを持つ。

長期保守では、package間依存DAGの自動検証、public APIとprivate APIの分離、init orderのテスト、optional module不在時のdegraded動作、contract test、schema migration、event replay test、snapshot互換性、source／derived／projectionの混同防止が重要になる。

## 11.3 セキュリティ上の優先事項

NBAccessではconfidential dependency propagation、CloudPublishable境界、held expression bypass、key indexのcross-kernel整合、MAC／tamper testが重要である。

Runtime／Orchestratorではadapter bypass、worker deny heads、committer capability、retry時の二重実行、external process cleanup、approval後のpolicy再検査が重要である。

SourceVaultではrequest-time release gate、revocation、private topic label漏洩、prompt／retrieval envelopeの差し替え、taint edge方向、heartbeat field漏洩、event replay決定性が重要である。

現時点で設計と実装の差分が確認されている優先項目は二つある。第一に、SourceVaultTaintDeclassifyがowner文字列確認単独で高リスク資料を解除できる点で、designのrisk別sanitizer／preview要件の実装が必要である。第二に、quarantineのSafetyStateがsearchindex・Knowledge Home・MCP・PromptRouterの共通gateへ未結線である点で、経路横断のquarantine不変条件検査が必要である（9.2.2・9.2.4・9.11参照）。

## 11.4 公開OSSとしての課題

公開リポジトリから見ると、GitHub Actions等の公開CI、test coverage指標、tag／release運用、version compatibility matrix、Windows以外の検証、reproducible setup、minimal sample vault、security modelの独立文書、threat model、migration guide、public changelogなどの整備が必要である。

## 11.5 推奨する段階的整理【将来提案】

以下は仕様上の確定順序ではなく、本レポート筆者による優先順位の提案である。

### 第1段階：四層の安定化

- NBAccess boundary test
- Runtime adapter contract
- Orchestrator commit test
- SourceVault core replay test

### 第2段階：Knowledge Home統合

- Mail、PDF、Notebook、Webの共通TopicPosition
- SharedAuthor／SharedTag edge
- unified NeighborhoodSearch
- cross-media live view

### 第3段階：安全境界の強制移行

- LLM boundary ShadowからWarnへ
- 対象entrypointのEnforce
- CapabilityLeaseとdispatchの一体化
- sink field allowlist

### 第4段階：長期知識の評価

- retrieval episodeの効果測定
- OOPS seedのrecall／NDCG評価

- novel topic成長率
- false link／false topic率
- cognition supportのfalse intervention率

## 11.6 今後の展望

短期的には、四層の boundary test と公開CIを最優先にするのが合理的である。NBAccess privacy、Runtime adapter、Orchestrator commit、SourceVault replayという中核境界を固定し、その上で機能追加を行う方が長期的な回帰を減らせる。

中期的には、Knowledge Home全媒体統合、LLM boundary Enforce、CapabilityLease action 統合、comprehensive SystemDoctor auto-trigger、multi-PC WorkerPoolを進めることで、現在点在する先進機能が一つの運用体験へまとまる。

長期的には、公開benchmark、sample vault、reproducible setup、threat modelを整備し、研究コードから再現可能な基盤へ昇格させることが重要である。特に安全性の主張は、攻撃テストとfailure injectionを含む継続的検証で支える必要がある。

## 第12章 総合評価

四つのリポジトリ群は、「LLMをMathematicaから呼ぶツール」から、「Notebookを中心に知識を取得し、根拠を保存し、agentが推論し、workflowが実行し、その結果を再び知識として蓄積する環境」へ発展している。その独自性を生んでいるのは機能数ではなく境界設計である。秘密をNBAccess、単一turnをRuntime、複数agentとcommitをOrchestrator、source lifecycleと知識空間をSourceVaultへ分け、MCPやAutoTriggerで外部・自律実行へ広げても、権限とprovenanceの境界を維持しようとしている。

一方、完成品として一枚岩に出来上がっているわけではない。全媒体近傍検索、全面Enforce、WorkerPool failover、公開CIなどは今後の課題である。したがって評価は「設計思想が先進的で、主要部の実装がかなり進んだ研究開発基盤」とするのが妥当である。以下、12.1～12.2で基礎と拡張の評価を、12.3～12.4で技術的価値と制約を述べ、12.5で最終結論、12.6で長期展望をまとめる。

### 12.1 基本評価

四つのリポジトリは、当初から明確な責務境界を持っていた。

- NBAccess：Notebookと秘密の境界
- ClaudeRuntime：単一turnの実行核
- ClaudeOrchestrator：複数workerとcommit discipline
- SourceVault：source lifecycleとdeterministic replay

特に、workerのNotebook直書きを禁止してsingle committerへ集約する設計、PromptRouterが未評価式を返すだけで自ら評価しない設計、Runtimeが安全判断をNBAccessへ委譲する設計は、実運用上の事故を減らすために一貫している。

### 12.2 マイニング・安全レイヤーを含めた評価

SourceVaultマイニングレイヤー（第8章）の追加により、システムは資料を保存・検索するだけでなく、topic、identity、引用、session、作業履歴によって知識を再編成できるようになった。

OOPSコーパスは、単なる過去メールではない。約6,500通のメールに、人手で構築された約4,100件のtopic item、topic relation、mail-to-topic、quote relationが付随する。この構造が、新しい一般メール、Notebook、PDF、Webページを位置づけるための初期座標になる。

同時に、多層防御レイヤー（第9章）は、知識の接続範囲が広がることで生じる危険を制御する。

- privacyはNBAccessとrelease contextで守る
- Prompt Injectionはuntrusted dataとtaintで隔離する
- LLM送信はPreparedInputTokenでbindする
- 外部actionはCapabilityLeaseで制限する
- Notebook commitはsingle committerへ限定する
- 診断情報はsink allowlistで制限する
- 認知支援はlocal-only、shadow-firstにする
- 異常分析はobserve-onlyから始める

この構造により、検索結果として近いこと、閲覧できること、LLMへ送信できること、外部操作を実行できることが分離される。

## 12.3 技術的価値

第一は、機密NotebookとLLMの安全な接続である。Notebookの秘密セルと依存出力を識別し、送信先ごとにcontextを変え、LLM proposalを保持式として検証する。

第二は、source-centricな再現可能性である。使用したsource、snapshot、span、claim、workflow、generated artifactを追跡し、上流資料の更新時に成果物のstalenessを判断する。

第三は、個人Knowledge Homeである。現在の資料から、topic、人物、引用、時間、過去の探索を手掛かりに次の知識へ移動できる。

## 12.4 制約

一般的なPython agentの代替を求める場合、LangGraphやHaystackなどの方が導入しやすい可能性が高い。transreal stackはMathematica、Wolfram Language、Notebook、複数パッケージのロード、local vaultを前提とする。

また、公開CI、release、platform validationの成熟度は限定的である。全媒体横断のKnowledge Home、安全境界の全面Enforce、認知・異常機能の実運用効果についても、今後の統合と評価が必要である。

## 12.5 最終結論

本システムは、完成済みの汎用エージェント製品というより、次のように評価するのが適切である。

NBAccess、ClaudeRuntime、ClaudeOrchestrator、SourceVaultは、Mathematica Notebookを核として、機密境界、LLM推論、複数agent、source lifecycle、知識グラフ、近傍探索、実行権限、監査を統合する、高度で実装の進んだ研究開発基盤である。そして四つのリポジトリは、「LLMをNotebookから呼び出すためのツール群」という範囲を超え、知識の取得、構造化、探索、推論、実行、監査を一つの責務分離モデルで管理するシステムへ発展している。

その独自性は、個別機能の数ではなく、次の全体を一つの連続したモデルとして扱う点にある。

```
graph TD
    Notebook --> Privacy
    Privacy --> LLM_Proposal[LLM Proposal]
    LLM_Proposal --> Validation
    Validation --> Workflow
    Workflow --> Commit
    Commit --> Source_and_Evidence[Source and Evidence]
    Source_and_Evidence --> Topic_and_Identity[Topic and Identity]
    Topic_and_Identity --> Neighborhood
    Neighborhood --> Diagnostics_and_Audit[Diagnostics and Audit]
```

これは、Wolfram Notebookを単なる計算文書ではなく、長期的な知識形成、推論、実行、検証の作業環境として再構成する試みである。

## 12.6 今後の展望

今後このシステムが最も大きく発展する可能性があるのは、SourceVaultを共有する複数のLLM・複数PC・複数workflowが、同じsv:// object graphとpolicyを共有する構成である。人間はNotebookを主要UIとして使いながら、背後ではlocal LLM、cloud LLM、remote worker、scheduled workflowが協調する。

その未来像を安全に実現するには、PreparedInputToken、CapabilityLease、single committer、release gate、taint provenanceを「オプション機能」ではなく実行基盤の標準経路として完成させる必要がある。自律性を増すほど、権限分離と監査性を強くするという逆方向の設計が必要になる。

最終的には、Wolfram Notebookを単なる計算文書ではなく、個人または研究グループのKnowledge Operating Environmentとして扱う構想へつながる。source、会話、コード、仕様、workflow、メール、文書、実験結果が同じ参照空間で結ばれ、それぞれのprivacyとprovenanceを保ったまま再利用されることが、この四リポジトリ群の長期的な到達点と考えられる。

## 付録A 主な参照資料

---

本レポートは、主として2026年7月15日時点のtransreal公開GitHubリポジトリのREADME、API文書、設計仕様、主要実装ファイルを基に整理した。特に第8章・第9章では以下を重点的に参照した。

- NBAccess/README.md
- NBAccess.wl
- NBAccess\_crypto.wl
- NBAccess\_info/design/ClaudeEval\_runtime\_orchestrator\_nbaccess\_async\_compat\_final\_spec\_permission\_modes\_v3.md
- ClaudeRuntime/README.md
- ClaudeRuntime.wl
- ClaudeRuntime\_processsupervisor.wl
- ClaudeRuntime\_info/docs/api\_processsupervisor.md
- ClaudeOrchestrator/README.md
- ClaudeOrchestrator.wl
- ClaudeOrchestrator\_workflow.wl
- ClaudeOrchestrator\_info/docs/api.md
- SourceVault/README.md
- SourceVault\_core.wl
- SourceVault\_searchindex.wl
- SourceVault\_identity.wl
- SourceVault\_mining.wl
- SourceVault\_oopsseed.wl
- SourceVault\_mailstructure.wl
- SourceVault\_knowledgehome.wl
- SourceVault\_diagnostics.wl
- SourceVault\_capbroker.wl
- SourceVault\_taint.wl
- SourceVault\_cognition.wl
- SourceVault\_adjudication.wl
- SourceVault\_anomaly.wl
- SourceVault\_info/design/sourcevault\_search\_foundation\_implementation\_spec\_v1.md
- SourceVault\_info/design/sourcevault\_self\_organizing\_mining\_identity\_tag\_spec\_v0\_1.md
- SourceVault\_info/design/sourcevault\_cane\_knowledge\_home\_mining\_spec\_v0\_7.md
- SourceVault\_info/design/sourcevault\_cane\_handoff\_2026-07-14.md
- SourceVault\_info/docs/api\_knowledgehome.md
- SourceVault\_info/docs/api\_oopsseed.md
- SourceVault\_info/docs/api\_identity.md
- SourceVault\_info/docs/api\_cognition.md

実装状況に関する記述は、設計仕様上の意図と、公開コード・API文書・引き継ぎ資料で確認できた実装状態を区別して記載した。

# 付録B 増補・改訂で参照した主要コード・API・設計文書

本増補では、READMEだけでなく、2026年7月16日時点の公開GitHub上の実装コード、SourceVault\_info/docsのAPIリファレンス、SourceVault\_info/designの仕様、コード化workflowの実装仕様を照合した。以下は追加記述の主な根拠である。

| 領域                               | 主要参照ファイル                                                                                                                                         |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| claudecode基礎層                    | transreal/claudecode/claudecode_info/docs/api.md; claudecode_info/docs/api_directives.md                                                         |
| PromptRouter                     | SourceVault_info/docs/api_promptrouter.md; SourceVault_promptrouter.wl                                                                           |
| PromptWorkflow                   | ClaudeOrchestrator_promptworkflow.wl                                                                                                             |
| 要件→仕様                            | SourceVault_workflows/spec-review/SVWorkflow_SpecReview_info/design/implementation-spec.md                                                       |
| 仕様→実装                            | SourceVault_workflows/spec-impl/SVWorkflow_SpecImpl_info/design/implementation-spec.md                                                           |
| Workflow registry                | SourceVault_info/docs/api_workflowregistry.md                                                                                                    |
| SourceVault実利用                   | SourceVault_info/docs/user_manual.md                                                                                                             |
| sv:// URI/MCP                    | SourceVault_info/docs/api_mcp.md; SourceVault_mcp.wl;<br>SourceVault_info/design/sourcevault_universal_mcp_access_spec_v2.md                     |
| Capability Broker                | SourceVault_info/docs/api_capbroker.md; SourceVault_capbroker.wl                                                                                 |
| AutoTrigger                      | SourceVault_autotrigger.wl; SourceVault_info/docs/api_autotrigger.md;<br>SourceVault_info/design/sourcevault_auto_trigger_scheduler_spec_v0_1.md |
| Diagnostics/SystemDoctor         | SourceVault_info/docs/api_diagnostics.md; SourceVault_diagnostics.wl                                                                             |
| RuntimeSession/Artifact boundary | ClaudeOrchestrator_info/design/claude_orchestrator_runtime_session_episode_petri_spec_v0_1.md                                                    |

Routine Attention／Agenda関連では以下を参照した。

- SourceVault\_info/design/sourcevault\_routine\_attention\_spec\_v0\_4.md — Routine Attention coreの正準仕様。v0.3は歴史参照。
- SourceVault\_info/design/sourcevault\_routine\_planview\_ext\_spec\_v0\_1.md — 計画層・可視化層extension。
- SourceVault\_info/design/sourcevault\_routine\_mail\_agenda\_spec\_v0\_1.md — R9 Mail Agenda仕様。
- SourceVault\_info/docs/api\_routine.md / SourceVault\_routine.wl — Obligation、Attention、R7 anomaly stream、R8 automation。
- SourceVault\_info/docs/api\_routineplan.md / SourceVault\_routineplan.wl — Plan、ScheduleFabric、Agenda、可視化。
- SourceVault\_info/docs/api\_mailagenda.md / SourceVault\_mailagenda.wl — actionable mail、resolution state、mail↔Notebook継承。
- 実装確認に用いた公開GitHub履歴上のcommit: 409012e (R7/R8/AT-1/routineplan core)、c0c487d (Agenda/visualization/ScheduleFabric/replan)、c8d6e82 (R9 mailagenda統合)。

## 実装状況の読み方

本レポートでは、仕様書に存在することと現行コードで実装されていることを区別した。特に、(1) comprehensive SystemDoctorを「最初の自動workflow」にする方針はAutoTrigger設計仕様上の正準だが、自動triggerは診断APIで延期項目として残る、(2) SpecificMachineによるリモート実行は実装済みだがWorkerPool自動負荷分散/failoverは未実装、(3) sv://は外部参照の正準identity層だがlegacy internal refは互換のため残る、(4) quarantineのSafetyState gateとrisk別declassificationは設計仕様が実装に先行している、という点は過大評価を避けるため明示的に区別している。

なお、claudecode\_coreへの将来分離については公開コードで実装済みの事実としてではなく、claudecode.wlの現行責務をコアとlegacy互換へ整理する筆者の整理案として記述した。



## 付録C ソースコードとAPI文書の原本間差分（2026年7月17日確認）

本レポートの照合過程で、ソースコードとAPI文書が食い違っている箇所が見つかった。レポート本文はソースコードを正としたが、原本同士の差分としてここに明示する。API文書だけを読むと実装状態を誤認する可能性がある。

| 項目                        | ソースコード（正）                                      | API文書の記述                       | 推奨対応                                |
|---------------------------|------------------------------------------------|--------------------------------|-------------------------------------|
| NBAccess crypto backend   | SystemCredentialのput/get/delete/index永続化を実装済み  | api_crypto.mdは「書き込みAPIはTODO」   | API文書とソース先頭コメントを更新                  |
| NBStoreCredentialKey 戻り値  | Status/KeyRef等のAssociation                     | keyRef/\$Failed                | 公開契約を決めて同期                          |
| NBDeleteCredentialKey 戻り値 | Status/KeyRefのAssociation                      | Boolean                        | 公開契約を決めて同期                          |
| Orchestrator並列Spawn       | MaxParallelism / UseDAG によるDAG並列経路を実装          | 「直列/順次、現状1」                    | optionと既定値を併記                       |
| R9 Mail Agenda            | SourceVault_mailagenda.wl と routineplan結線が実装済み | api_routine.md末尾に「R9=mail 未実装」 | 文書を最新mainへ追随                        |
| Declassification          | owner文字列確認だけで解除可能（API文書も同様）                    | designはrisk別要件を規定              | designのrisk別要件を実装するか、design側に未実装と明記 |